linnest

a typst interface to the `linnet` crate

`linnet` is a Rust crate that provides a graph data structure, graph algorithms, several layout algorithms, and its own DOT reader and writer.

Linnest Typst API

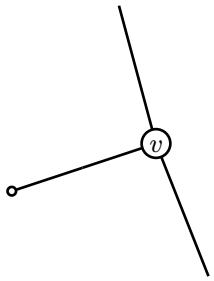
Contents

Linnest Typst API . . . 1
- Linnet . . . 2
- Linnest . . . 2
  - Choose an import path . . . 3
  - Minimal Build Example . . . 3
  - Graph Objects . . . 4
  - Graph object API . . . 4
  - Graph Specs . . . 5
  - Placements . . . 7
  - Drawing . . . 8
  - Domain Styles And Templates . . . 9
  - Graph Queries . . . 14
  - Default diagram layout . . . 15
  - Spring and annealing layout model . . . 16
  - Subgraphs . . . 21
  - Directed Cut Views . . . 21
- Generated Reference . . . 23
- graph . . . 23
  - from-spec . . . 23
  - build . . . 23
  - parse . . . 26
  - node . . . 33
  - source . . . 34
  - sink . . . 35
  - edge . . . 36
  - group . . . 37
  - pin . . . 38
  - start . . . 38
  - pos . . . 38
  - map . . . 40
  - style . . . 41
  - eval-fields . . . 43
  - info . . . 44
  - dot . . . 45
  - nodes . . . 45
  - edges . . . 46
  - node-data . . . 46
  - edge-data . . . 46
  - update-node-data . . . 47
  - update-edge-data . . . 48
  - cut . . . 48

boundaries . . . 49
join . . . 50
join-edges . . . 50
cycles . . . 51
forests . . . 51
Reference . . . 52
options . . . 52
layout . . . 52
layout-sequence . . . 65
Reference . . . 65
edge-halves . . . 65
to-cetz-edge-halves . . . 66
draw . . . 68
subgraph . . . 79
label . . . 79
bits . . . 80
compass . . . 80
to-label . . . 81
hedges . . . 81
contains . . . 82
select . . . 82
with-data . . . 83
hedge-data . . . 84
complement . . . 84
contains-edge . . . 85
node-depths . . . 85

Linnet

The `linnet` crate wrapped by this package is built around a half-edge graph data structure. This means that instead of a graph being represented as a set of nodes and edges, it is represented as a set of half-edges H, and a set of vertices V. The graph structure is then encoded through two maps. The first map, $\partial : H \longrightarrow V$ maps each half-edge to its corresponding vertex. The preimage of any vertex v is the set of half-edges that map to it, called the crown of v. The second map, $\iota : H \longrightarrow H$, is an involution that *glues* half-edges together to form edges. If a half-edge is glued to itself, we call that an external half-edge. This means that linnet graphs are strictly more capable than normal edge and vertex graphs.



Native half-edges also make subgraphs more granular because they can be encoded as sets of half-edges. This directly supports vertex-induced subgraphs: the union of the crowns of a set of vertices.

Linnest

Linnest is the Typst and WebAssembly interface to Linnet. It provides layout algorithms, DOT parsing, and selected graph algorithms without a separate runtime process.

Graphs can be constructed in two ways: parse a DOT string with `parse`:

```
#let g = parse("digraph { a -> b }")
```

or build from edges and nodes with `build`, using a Fletcher-inspired syntax:

```
#let g = build({
  node(<a>, label: [$v$])
  node(<b>)
  edge(source(<a>), <a-b>, sink(<b>), label: [e])
  edge(source(<a>), <in-a1>, label: [e])
  edge(source(<a>), <in-a2>, label: [e])
})
```

In either case, `type(g)` is `dictionary`: graph values wrap an archived Linnet graph together with native Typst data. Rust owns topology, layout state, statement metadata, and internal opaque payload bytes. User data captured from Typst stays in Typst and is merged back into query records.

The main use case is to place nodes and edges on a canvas with the `layout` function and render the result with `draw`.

- `graph` for construction, parsing, inspection, directed cuts, joins, and graph algorithms.
- `subgraph` for half-edge selection, annotations, and inspection.
- `layout` for the separate layout pass.
- `draw` for rendering a laid-out graph object with CeTZ.

Domain-specific styles are ordinary Typst data and callbacks composed with `graph.style` and `draw`; Linnest does not reserve a domain or export a physics module.

Choose an import path

Linnest and Kurvst are currently bundled source packages, not Typst Universe packages. A Clinnet run writes both package trees below `build/templates/`. From a custom template in that directory, import Linnest with:

```
#import "crates/linnest/typst/src/lib.typ": draw, graph, layout, subgraph
```

From this repository's `crates/linnest/typst/examples/` directory, the equivalent checkout-relative import is `../src/lib.typ`. Keep the package directory and its `linnest.wasm` file together when copying it elsewhere. The examples below use the checkout-relative form because they are also compiled as repository tests.

Minimal Build Example

```
#import "../src/lib.typ": draw, graph, layout, subgraph
#import graph: build, dot, edge, edges, node, nodes, parse, sink, source

#let g = build({
  node(<a>)
  node(<c>)
  edge(
    source(<a>, compass: "e"),
    <a-c>,
    sink(<c>, compass: "w"),
    label: [a-c],
    statements: (
      color: "0055ff",
      source-color: "d72638",
      sink-color: "1b7f4c",
    ),
  )
},
  name: "demo",
)
#let g = layout(g)
#let east = subgraph.compass(g, "e")
#let edge-records = edges(g, subgraph: east)
#let dot-text = dot(g)
#let edge-label(edge) = text(fill: rgb("#" + edge.color))[#edge.label]
```

```
#let source-style(edge) = (stroke: rgb("#" + edge.source-color) + 0.5pt)
#let sink-style(edge) = (stroke: rgb("#" + edge.sink-color) + 0.5pt)
#html.frame(draw(
  g,
  subgraph: east,
  edge-label: edge-label,
  edge-label-style: (anchor: "south"),
  source-style: source-style,
  sink-style: sink-style,
))
```



Graph Objects

Graph values combine archived Linnet topology with native Typst data. The focused graph reference documents their constructors, transforms, queries, and update operations.

Graph object API

Graph objects are Typst dictionaries wrapping archived Rust graph bytes plus native Typst data arrays for graph, node, edge, source, and sink data. The Rust graph may also carry an internal opaque payload, but that payload is used only by the Typst wrapper and is not exposed in public records. Build or parse graph objects with `graph`, transform graph objects with `layout`, and pass objects back to `graph` or `subgraph` for inspection. Subgraphs are also dictionaries: they combine an archived half-edge selection with a topology signature and Typst-only annotations, rather than exposing raw bytes as public inputs.

- `graph.parse(input)` parses one or more DOT digraphs and returns an array of graph objects. Its `eval-graph-fields`, `eval-node-fields`, `eval-edge-fields`, `eval-source-fields`, and `eval-sink-fields` arguments are convenience arguments for `graph.eval-fields`.
- `graph.build(..)` constructs one graph object from a stream of node and edge items.
- `graph.map(graph, ..)` maps graph, node, edge, source, and sink records to new native data or structural patches without changing topology. `node` and `edge` accept a callback or a dictionary keyed by current record names. Each dictionary value is a patch, a callback receiving the full record, or `none`. Edge patches may set `spring-length` directly; it is validated and stored as the layout statement. Unlisted records remain unchanged; unknown names are errors.
- `graph.cut(graph, left: left, right: right, boundary: patch)` opens a weighted directed cut into a new graph, preserving Typst data and recording origins. `graph.boundaries(view)` queries its current boundary endpoints.
- `graph.node-data(graph, <name>)` and `graph.edge-data(graph, <name>)` return one named node or edge data.
- `graph.update-node-data(graph, <name>, data)` and `graph.update-edge-data(graph, <name>, data)` update one named node or edge data. Direct replacements and callbacks run in Typst with `(data, record)`.
- `graph.eval-fields(graph, ..)` evaluates selected record fields into data entries. It works on parsed and built graph objects.
- `node(..)` returns a node item.
- `source(..)` and `sink(..)` return half-edge endpoints.
- `edge(..)` returns an edge item built from source/sink endpoints and an optional edge name.
- `layout(graph, ..)` runs layout as an explicit second step. Its settings are named parameters so calls stay descriptive and Tidy can document each field.
- `draw(graph, ..)` draws a laid-out graph object with CeTZ.
- `dot(graph)` returns a DOT string for inspection or export.

Graph Specs

The Typst construction API is half-edge first: create node items, create source and sink half-edge endpoints, then pass edge items to `graph.build`. `graph.build` accepts both comma-separated items and ordinary Typst code blocks. Typst labels such as `<a>`, `<h1>`, and `<e1>` are API names; node and edge names are emitted back from `nodes(g)` and `edges(g)` as Typst labels. They are resolved before the wire format is sent to the Rust plugin. Numeric `id` arguments choose graph indexes or ordering: on nodes, `id` fixes the resulting node index and must be unique and in bounds. On nodes, edges, sources, and sinks, extra named arguments are captured as opaque Typst data fields: `edge(source(<a>, style: (stroke: red + 0.7pt)), sink(<b>), kind: "link")` stores `(style: ..)` in the source data and `(kind: "link")` in the edge data. These user data values are not sent through the Rust plugin boundary. Typst sends an internal opaque payload with correlation data, asks Rust to resolve the graph indexes, then stores user data in arrays at those resolved graph, node, edge, and half-edge ids. `graph.info`, `graph.nodes`, and `graph.edges` merge those native values into the returned records. A captured `label` data field on nodes and edges is display content used by the default drawing style; use `statements: (label: "...")` when a flat metadata label string is needed. Statements are flat metadata used by DOT; they cannot nest. Values are scalar strings/numbers/booleans. Use data fields for structured Typst data or content.

`graph.build` and `graph.parse` also accept `default-node-data`, `default-edge-data`, `default-source-data`, and `default-sink-data`. These defaults are merged into the corresponding data; captured data fields on nodes, edges, sources, and sinks override the defaults. Drawing callbacks decide how those fields affect the result. For example, endpoint styles can read `data.style` from the source and sink half-edge records:

```
#let g = build({
  node(<a>, label: [a])
  node(<b>, label: [b])
  edge(
    source(<a>, style: (stroke: red + 0.7pt)),
    <e>,
    sink(<b>, style: (stroke: blue + 0.7pt)),
    label: [connection],
    kind: "dependency",
  )
},
  default-edge-data: (kind: "propagator"),
)
#let endpoint-style(edge, side) = {
  let endpoint = edge.at(side + "-half-edge", default: none)
  if endpoint == none { return (:) }
  endpoint.data.at("style", default: (:))
}
#draw(
  layout(g),
  source-style: edge => endpoint-style(edge, "source"),
  sink-style: edge => endpoint-style(edge, "sink"),
  edge-label: edge => edge.at("label", default: none),
)
```

When parsing DOT, the same native data arrays can be filled from selected string fields. Rust parses DOT into topology and statement metadata; Typst applies defaults and evaluates selected fields afterward. The selected fields are evaluated with the record's merged `fields` dictionary in scope. Since node names are labels, use `#str(name)` when a name should become visible text:

```
#let g = parse(
  "digraph g { a [label=\"A\"]; a -> b [label=\"$p$\", source=\"out\", sink=\"in\"] }",
  eval-node-fields: ("label",),
  eval-edge-fields: ("label",),
  eval-source-fields: ("statement",),
  eval-sink-fields: ("statement",),
).first()
```

```
#nodes(g).first().data.label
#edges(g).first().data.label
```

The same transform can run after construction. This is useful for global edge statements that should apply to every edge while still seeing local edge fields:

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>), statements: (mom: "p"))
}, default-edge-statements: (display-label: "$#mom$"))
#let g = graph.eval-fields(g, eval-edge-fields: ("display-label",))
#edges(g).first().data.at("display-label")
```

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(source(<a>), <e1>, sink(<c>))
})
```

Named nodes and edges can be updated after construction without scanning in the caller. The update replaces the native data, or it can be a callback receiving `(data, record)`:

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(source(<a>), <e1>, sink(<c>))
})
#let g = update-node-data(g, <a>, (label: [A]))
#let g = update-edge-data(g, <e1>, (data, edge) => (
  label: [$p$],
  source: edge.source.node,
))
#node-data(g, <a>).label
#edge-data(g, <e1>).label
```

`source(..)` and `sink(..)` accept a node reference plus optional `name`, `id`, `statement`, and `compass`. `name` is a Typst label name for the half-edge; `id` is numeric:

```
edge(source(<a>, name: <h1>, id: 0), <e1>, sink(<c>, id: 2), label: [a-c])
```

One half-edge creates an external edge. The side is determined by the constructor, so there is no public `flow` argument:

```
edge(<incoming>, sink(<a>))
edge(source(<c>), <outgoing>)
```

`graph.edge(..., spring-length: 1.5)` multiplies that edge's existing preferred spring length by `1.5` in force/anneal layout. The factor must be positive, finite, and dimensionless; it applies to each node-to-control-point spring, including the existing doubled rest length for dangling edges. The argument defaults to `none`, preserving local or default `spring-length` statements; a missing statement means `1`. An explicit argument overrides those statements. This is not a guaranteed rendered length, a rendering option, or a change to global layout `spring.length`; tree/dot/stable-layered layout is unaffected.

`graph.build` does not interpolate statement strings on the Rust side. Use `graph.eval-fields` or `graph.map` when a default statement should turn into Typst content or structured data. The evaluation scope includes the record's merged fields, so default edge statements can still refer to local edge fields:

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(
    source(<a>),
    <a-c>,
    sink(<c>),
    label: [a-c],
```

```
    statements: (color: "0055ff", label: "a-c"),
  )
},
  default-edge-statements: (
    color: "000000",
    display-label: "$#label$",
  ),
)
#let g = graph.eval-fields(g, eval-edge-fields: ("display-label",))
```

Placements

`graph.pos` creates a first-class placement. The default `mode: "pin"` turns a coordinate into a fixed layout constraint and, for x/y, a drawable position. Use `mode: "start"` when the coordinate should only seed the layout, or use `graph.pin(value)` / `graph.start(value)` to choose a mode for one axis:

```
#let g = build({
  node(<a>, pos: graph.pos(x: -2, y: 0))
  node(<c>, pos: graph.pos(ref: <a>, dx: 4, dy: 0))
  edge(source(<a>), <a-c>, sink(<c>), pos: graph.pos(x: 0, y: 1.2))
})
```

Auxiliary `z` is force-layout depth in the same numeric layout units as x/y, not a Typst length or a rendered coordinate. Both `graph.build` and `graph.map` accept `graph.pos(z: graph.pin(2))` for a hard raw-depth pin and `graph.pos(z: graph.start(2))` for a movable initial depth. A bare `graph.pos(z: 2)` follows `mode`, defaulting to `"pin"`:

```
#let g = graph.map(g,
  node: node => (pos: graph.pos(z: graph.pin(2))),
  edge: edge => (pos: graph.pos(z: graph.start(-1))),
)
```

A z-only placement leaves XY coordinates, constraints, and partial-position flags alone, including automatic edge midpoint seeds. Omitting z preserves existing depth metadata during unrelated XY patches. `ref`, `dx`, and `dy` affect XY only; z groups, relative depth references, `dz`, and non-finite z values are not supported.

Node and edge `statements` expose `"pos-z"` as finite numeric text and `"pos-z-mode"` as `"pin"` or `"start"`. These describe the supplied raw pin or seed; output `pos` remains a two-coordinate x/y record. The force solver multiplies raw depth by a global scale that flattens to zero: a hard raw pin remains fixed even when its effective depth is zero. Other layout backends do not use z. Auxiliary depth does not change draw order or provide an over/under rendering layer.

`graph.group` links one XY coordinate across several nodes or edge control points. A `side` of “+” keeps the coordinate positive, and “-” keeps it negative. Use `start` to seed the shared coordinate without fixing it. Matching node and edge groups are one degree of freedom during force and annealing layouts, so repulsion and springs act on their combined force rather than being reconciled afterward. If several members supply starts for one group, their mean initializes the shared coordinate. GammaLoop external-edge columns use this to keep incoming and outgoing external legs on opposite sides while pairing rows by a shared `y` group:

```
#let edge-items = (
  edge(
    source(<right-ext>),
    sink(<center>),
    pos: graph.pos(x: graph.group("right", side: "+", start: 4), y: graph.group("edgee0")),
  ),
  edge(
    source(<center>),
    sink(<left-ext>),
    pos: graph.pos(x: graph.group("left", side: "-"), y: graph.group("edgee0")),
  ),
)
```

Raw DOT input uses the same placement model through the `pos` attribute. The standard Graphviz subset is preserved: `pos="x,y"` is a starting coordinate and `pos="x,y!"` is pinned. Linnest extends the value with explicit id references and axis entries. In axis entries, `!` belongs to that axis: numeric entries without `!` are starting coordinates, numeric entries with `!` are fixed constraints, and grouped entries must use `!` because groups are constraints.

```
digraph {
  a [id=0 pos="0,0!"]
  b [id=1 pos="ref(node:0)+4,0!"]
  a -> b [id=0 pos="ref(node:1)+0,1!"]
  b -> c [id=1 pos="ref(edge:0)+1,0!"]
  c [id=2 pos="x:2!"]
  d [id=3 pos="x:2!,y:1"]
  ext -> a [id=2 pos="x:@-left!,y:@edge0!"]
}
```

The DOT grammar is intentionally id-based: `ref(node:0)` uses a node `id` and `ref(edge:0)` uses an edge `id`. Bare names and implicit edge order are not placement references. The `@` syntax denotes grouped coordinate constraints; `@+name` and `@-name` keep the grouped coordinate on the positive or negative side respectively.

```
pos="x,y"                   // start x and y
pos="x,y!"                  // pin x and y
pos="x:<coord>"             // start numeric x
pos="x:<coord>!"            // pin x
pos="y:<coord>!"            // pin y
pos="x:<coord>!,y:<coord>"  // pin x, start numeric y
pos="x:<coord>,y:<coord>!"  // start numeric x, pin y
```

Drawing

Draw styling is Typst-native. Pass dictionaries or callbacks to `draw`; edge callbacks receive the merged `scope`, edge statements, source/sink half-edge records, and the edge index:

```
#let edge-label(edge) = text(fill: rgb("#" + edge.color))[#edge.display-label]
#let source-style(edge) = (stroke: red + 0.5pt)
#let sink-style(edge) = (stroke: blue + 0.5pt)
#draw(layout(g), edge-label: edge-label, source-style: source-style, sink-style: sink-style)
```

Edge labels are drawn with CeTZ content at the layout label position. Use `edge-label-style: (anchor: "south")`, or an edge-data callback returning a style dictionary, to choose which point of the label is anchored there.

Add overlays with `draw-after: (g, bounds) => { ... }`. The callback receives the positioned graph and its rendered bounds before overlays and canvas padding. The numeric `left`, `right`, `top`, `bottom`, `width`, and `height` fields use graph units; `top` is the greatest y coordinate. Bounds include drawn shapes, curves, and label boxes. Ordinary CeTZ paths exclude extra stroke thickness; Kurvst heads contribute their painted outlines so thick caps and miters cannot escape the canvas. An empty drawing supplies zeros for every field. Overlay elements share `unit` and can enlarge the final canvas without affecting graph layout. `draw-after` also accepts an array of CeTZ elements or `none`.

```
#draw(layout(g), draw-after: (g, bounds) => {
  cetz.draw.rect(
    (bounds.left - 0.2, bounds.bottom - 0.2),
    (bounds.right + 0.2, bounds.top + 0.2),
    stroke: gray,
  )
})
```

Marks can follow graph orientation as a first-class draw option. Put the same mark layer on both halves and set `mark-orientation: "edge"`; default-oriented edges mark the source half, reversed edges mark the sink half with the marker flipped, and undirected edges suppress the mark. Use `mark-position: "center"` or a numeric ratio from `0` to `1`, then `mark-shift` for a signed arc-length adjustment along the derived path.

This positioning is independent of whether the layer is straight or patterned. `"center-if-dangling"` centers a dangling mark but keeps an orientation-selected paired mark at the source/sink split point. Pass data-only `curve.mark` specifications; legacy CeTZ mark dictionaries are errors. Stations refer to the visible carrier after offsets, node outsets and crossing windows, before arrow shortening or bending. Interior marks straddle the station on a chord. At ends, `fit: "chord"` preserves the carrier and trims the shaft to the head's back contact. `fit: "bend"` instead retracts the endpoint along its tangent while retaining control points, with `shorten` scaling the retraction (default 100%; 0% leaves the endpoint unchanged). Short carriers keep the full-sized head and clamp the shaft contact separately. Patterns, footprints and collision checks use the resulting painted shaft. `curve.mark.combine` fits all children and gaps as one composite.

Preparation callbacks and named path anchors refer to the derived, unshortened reference carrier. Fitted heads and shafts are produced after placement selection, so preparation callbacks must not depend on fitted-head geometry. Use `draw-after` to inspect or decorate the final painted drawing. Callbacks are not evaluated again to resolve head-dependent state.

```
#let oriented-arrow = (
  stroke: black + 0.7pt,
  mark: curve.mark.triangle(length: 0.15cm, width: 0.1125cm),
  mark-position: "center-if-dangling",
  mark-orientation: "edge",
)
#draw(layout(g), source-style: oriented-arrow, sink-style: oriented-arrow)
```

Domain Styles And Templates

Linnest's drawing layer is domain-neutral. A caller can map arbitrary node, edge, and half-edge data to reusable `draw` callbacks without changing topology or adding a domain-specific core option. The following map uses an application-defined `kind` field; the names and styles have no meaning to Linnest itself:

```
#import "../src/lib.typ": curve, draw, graph, layout
#import graph: build, edge, node, sink, source

#let kinds = (
  dependency: (stroke: rgb("#315f9f") + 0.8pt),
  event: (
    stroke: rgb("#a24b36") + 0.8pt,
    mark: curve.mark.triangle(length: 0.15cm, width: 0.1125cm),
    mark-position: "center",
    mark-orientation: "edge",
  ),
)
#let edge-style(edge) = kinds.at(edge.fields.at("kind", default: "dependency"))
#let g = build({
  node(<queued>, label: [queued])
  node(<running>, label: [running])
  node(<done>, label: [done])
  edge(source(<queued>), <starts>, sink(<running>), kind: "event", label: [starts])
  edge(source(<running>), <needs>, sink(<done>), kind: "dependency", label: [needs])
})
#let g = graph.style(g, edge-label: edge => edge.label)
#draw(layout(g, layout-algo: "stable-layered"), edge-style: edge-style)
```

The Feynman example style is configured before layout. `graph-style` is a function returning options for `graph.style`; `draw-style` remains a dictionary of drawing callbacks and defaults:

```
#import "../examples/map-style.typ" as feynman
#let styled = graph.style(g, ..feynman.graph-style(
  unit: 1.35, line-width: 0.7pt, node-radius: 0.2,
  momentum-line-width: 0.4pt,
))
#draw(layout(styled), ..feynman.draw-style)
```

Call `feynman.graph-style()` for the default appearance; replace the former dictionary spread `..feynman.graph-style` with `..feynman.graph-style()`. Its named options are:

- `unit: 1.35`: graph coordinate unit; a number multiplies the current `1em`, or supply an absolute length such as `10pt`.
- `line-width: 0.5pt`: ordinary particle stroke thickness.
- `node-radius: 0.08`: visible node radius in graph units.
- `node-line-width: auto`: node outline, defaulting to `line-width`.
- `massive-line-width: auto`: scalar/ghost stroke, defaulting to `2 * line-width`.
- `momentum-line-width: auto`: momentum shaft and arrowhead thickness, both defaulting to `0.8 * line-width`.
- `fermion-mark`: a Kurvst triangle at `0.10cm × 0.075cm`, filled by default.
- `momentum-mark`: a Kurvst straight head at `0.10cm × 0.075cm`. Either mark may be replaced with another `curve.mark` specification or `none`.

Width overrides are Typst lengths, independent of the graph unit and node radius. With all defaults, the node and ordinary edge strokes are `0.5pt`, scalar/ghost strokes `1pt`, and momentum strokes `0.4pt`. Mark stroke thickness comes from its carrier line; the former `fermion-arrow-line-width` option is removed. Choose shape, size and paint through `fermion-mark` instead. The constructor stores resolved `node-style`, `fermion`, `particles`, `momentum-stroke`, and `momentum-mark` presets in `scope.feynman`. `graph.style` uses this scope for measurement, and `draw` inherits it after layout; no repeated draw-time configuration is needed. Separate styles do not share configuration. Direct calls to the exported `feynman.node-style` and `feynman.edge-style` without scope still use the original defaults.

Particle aliases remain `a`/`photon`, `g`/`gluon`, and `scalar`/`ghG`; other particle names use the fermion preset. Hidden nodes always have radius zero and no fill or stroke. Per-edge momentum fields, including sparse `mom(...)` patches described below, retain priority over inherited edge data and keep their arrow/label placement independent of these width options.

Momentum visibility belongs to edge data: set `show-momentum: false` on an `edge(...)` or in a `graph.map` edge patch to omit its momentum arrow and label while retaining the propagator and fermion arrow. It defaults to `true`. Set `default-edge-data: (show-momentum: false)` on `graph.build` to hide all momenta, then opt individual edges back in with `show-momentum: true`.

Ordinary dangling-edge labels without a stored `label-pos` clear their external endpoint by `external-label-gap` (default `0.25` canvas units). Linnest measures the text box, including its anchor, padding and rotation, and moves it along the rendered carrier's outward endpoint tangent until the nearest projection clears that endpoint. This applies to direct `draw`, Python graphs, Feynman diagrams and process schematics. Existing explicit or layout-produced label positions and path-attached labels retain their placement. Set `draw(..., external-label-gap: 0.65)` in Typst, or `RenderConfig(drawing=DrawOptions(external_label_gap=0.65))` in Python, to change the clearance. The value must be non-negative; zero removes the extra gap while still placing the text box outside the endpoint.

Momentum arrows and labels are placed together by the shared label annealing step. The arrow offset and relative arc shift remain fixed; the optimizer chooses a position along the full edge and may switch an automatic side. It can move neighbouring annotations together to escape crowded placements. The complete arrow stays within the carrier. The shared physics renderer uses `momentum-arrow-offset: 0.35`, `momentum-arrow-length: 1.4`, and `momentum-arrow-ratio: 0.5`: the requested length is capped at half the offset carrier's arc length. Set the ratio to `none` for an absolute length without this fractional cap. Sparse `momentum(...)` patches configure the arrow and nested label options independently:

```
#import "../examples/map-style.typ": momentum
#let options = momentum(
  offset: 0.4, length: 1.0, shift: 0.2,
  label: (gap: 0.2, slide: true),
)
```

Use `label: (slide: false)` to pin the arrow and text at their requested positions, or `side: "left"` / `"right"` to keep the annotation on that side while still allowing arc movement. An explicit label anchor also pins it. Collision padding and the debugging boxes use the same settings as ordinary edge labels.

Cut fragments inherit the flag like other edge data. Use `..feynman.draw-style` unchanged; `edge-style` no longer takes a `show-momentum` argument. Only drawing layers are suppressed: physical momentum data is retained, and hidden layout labels still participate, so the switch does not change solved graph geometry.

An edge can sparsely patch the `edge-style` passed to `draw` with `style: (...)`, compute the patch with a callback, use `auto` to delegate, or use `none` to hide its paint while retaining topology. Local style callbacks receive the complete post-layout drawing record. Custom templates can package the same policy behind their mandatory `render(config)` export; the selected application owns the meaning and schema of any extra options.

Set `edge-offset` on `draw`, or `offset` in a `source-style`/`sink-style` dictionary, to draw a fitted parallel path. Style callbacks may also return an array of dictionaries; the layers are drawn in order on the same graph edge, so parallel strokes do not require duplicate graph edges.

```
#let base-style(edge) = (
  stroke: (paint: gray, thickness: 0.7pt, cap: "round"),
)
#let offset-style(edge) = (
  offset: 0.18,
  length: 1.4,
  ratio: 0.5,
  resolve-length: "min",
  shift: 0.2,
  stroke: (paint: rgb("#2f6f4e"), thickness: 1pt, cap: "round"),
  label: [active],
  label-side: "left",
  label-gap: 0.12,
)
#let source-style(edge) = (base-style(edge), offset-style(edge))
#let sink-style(edge) = (
  base-style(edge),
  offset-style(edge) + (mark: curve.mark.triangle()),
)
```

The parallel path is applied to the base edge geometry before patterns and other decorations; node outsets then trim the shifted path, so shifted paths still start and end outside fitted node circles. Add `edge-length` or `length` to center-trim the shifted path to a fixed arc length, and add `edge-ratio` or `ratio` to cap it by a fraction of the full offset path length. `edge-resolve-length`
`resolve-length` decides how to combine both limits `"min"`/`"shorter"` (default), `"max"`/`"longer"`, `"length"`/`"fixed"`, `"ratio"`/`"relative"`, `"none"`/`"full"`, or a function receiving `(offset-path-length, length, ratio)`. Lengths and shortening distances are measured after offsetting, since a curved parallel path need not have the same length as its centerline. For a finite layer, `shift` is an arc-length displacement along the complete offset path: positive values move toward the path end and values that would cross an endpoint are clamped. A layer can attach `label` content to its own path. `label-shift` (default `0`) moves the label's reference point by signed arc length from that derived path's midpoint, clamped to its endpoints: `clamp(path-length / 2 + label-shift, 0, path-length)`. Positive values move toward the path end, without moving or trimming the layer itself. `label-side` chooses `"left"` or `"right"` relative to the local path direction at the shifted point; `auto` follows the side selected by ordinary edge-label layout. With `label-style.anchor` omitted or set to `auto` (also `"auto"`), the label is centered and moved outward along the normal until its measured CeTZ box clears the finite attachment path by `label-gap`. This includes text bounds, wrapping, padding and rotation. Momentum text clears its finite arrow and also stays off its finite parent edge; when that edge binds, the arrow gap can be larger. Imaginary extensions beyond path endpoints do not repel text. An explicit anchor, including `"center"`, instead sits at the shifted reference point plus `label-gap` along the chosen normal, without any box-clearance correction. Negative gaps are

clamped to zero in both modes. Other `label-style` fields are forwarded to CeTZ content drawing. Clearance uses an adaptive approximation of the actual curves. Explicit anchors intentionally bypass this correction and may overlap the path. In `examples/map-style.typ`, `momentum-label-anchor` selects this behavior: omit it or use `auto` for box clearance, or choose e.g. `"east"` for direct anchor placement. `momentum-label-gap` supplies the gap in graph units. The momentum annotation uses the full offset carrier for candidate placement. `momentum-label-shift` defaults to the requested `momentum-arrow-shift`; independent values set the preferred relative arc separation. Automatic placement adds the same displacement to the arrow and label, preserving that separation. Its allowed range keeps the entire arrow inside the carrier, so changing the arrow length can change the pair's available positions. `momentum-label-slide: false` pins the requested placement instead. Collision checks use separate text boxes and rendered arrow shafts and heads. Fixed offset arrows are obstacles too. Ordinary edges contribute only where their painted paths intersect the measured text box, with no surrounding repulsion band. This intersection test excludes `label-collision-padding`; node boxes, neighbouring labels and self-loop bends retain their clearances. Stroke width uses a conservative envelope around the text box; dash gaps are treated continuously, so placement does not depend on the phase of a dash.

Import `momentum as mom` from `examples/map-style.typ` for compact, sparse options. `mom(side: "right", length: .7, shift: -.4, label: (gap: .2, shift: -.75, anchor: "south-west"))` expands to the existing `momentum-arrow-*` and `momentum-label-*` fields. Arrow options are `side`, `offset`, `length`, `ratio`, and `shift`; label options are `gap`, `shift`, and `anchor`. Only supplied options are emitted, so `mom(label: (gap: .2))` preserves an inherited arrow offset or label shift. Combine the result with other edge patches using dictionary addition, or spread it into `edge(..)`.

The attached label replaces the ordinary painted edge label, while that ordinary label may still supply the pre-layout size used by label layout and side selection; attached labels do not add a second collision constraint. When compatible source and sink layers request a centered mark, Linnest emits that mark once on the complete derived layer. The mark therefore follows `shift` instead of being centered separately on either half-edge. Set `offset-side: "label"` on an offset layer to choose the sign of `offset` so the layer is drawn on the same side of the curve as the edge label.

Pattern layers are generated over each continuous path before colour boundaries or gaps divide the painted stroke. Internal curve segments, source/sink colours, and crossing gaps do not restart the phase, shorten the endpoint taper, or change the fitted coil. Split positions follow distances on the underlying carrier, including when the decorated coil doubles back.

`pattern-natural-endpoints` defaults to `false`. For built-in coil strings (`"coil"`, `"helix"`, or `"spring"`) on complete continuous paths with both endpoints anchored, `true` constructs a fitted `kurvst.coil` dictionary in Typst from the path length and layer's pattern settings, then applies it once through ordinary `kurvst.pattern`. Its half-integer coil periods retain full amplitude, inward endpoint phases, and exact endpoints, without taper, stubs, or connectors. This overrides `pattern-phase`, bypasses `pattern-fit` integer fitting, and ignores `pattern-endpoint-slope` (`endpoint-ramp: false`).

Only automatic construction requires a built-in coil string. A fitted dictionary can be passed directly as `pattern` without this flag, with matching `pattern-*` settings; see the Kurvst manual's Path Patterns section for fitting and one-pass application. On curved carriers, fitted coils use local tangent/normal offsets, not evaluation at a corrected arc distance.

The gluon preset in `examples/map-style.typ` enables this option. Set `pattern-natural-endpoints: false` to restore its earlier 75%-wavelength endpoint taper (capped at half the path length), with a squared longitudinal envelope and no straight end sections. Without automatic fitting, `pattern-fit: true` adjusts `pattern-wavelength` to the nearest whole number of periods on a complete edge; `pattern-phase` is in radians, with `calc.pi / 2` giving coils matching endpoint phases. Set `pattern-endpoint-slope: 1` on a layer to allow an angled endpoint instead of a tangential one; the default is `0` and the valid range is `0` to `3`. This controls the taper envelope, not an angle, so the final direction also depends on phase, amplitude, and wavelength. Endpoints remain attached.

Source and sink styles with the same carrier and pattern geometry share this complete decoration even when their paints differ. Different patterns or carrier offsets define independent decorations on the two halves. For a shared pattern with `offset-side: "label"`, the full carrier selects the offset side once; both halves and their marks use that same signed offset.

Paired edges are Kurvst paths split at their edge layout point. Set `edge-split-gap` on `draw` to open a centered arc-length gap there, or set `split-gap` on an individual source/sink style layer to override the global value for only that layer. Wave and coil phases continue across the hidden span. `edge-halves` and `to-cetz-edge-halves` accept the same `split-gap` control and report the effective gap if either half is too short. This cuts at `edge.pos`; it does not detect arbitrary crossings.

For a known centerline crossing, put `crossing-under: 5` (using the target's integer edge id) or `crossing-under: <bridge>` (using its Typst edge name), and optionally `crossing-gap` (default `0.55`) on the style layer that should be interrupted. Targets are resolved independently of edge order. Kurvst locates proper intersections on the carrier and removes the corresponding intervals from the finished decoration. This works for dangling layers and for paired layers whose source and sink share carrier and pattern geometry; their paints may differ. Both paired half styles must name the same target and gap. A cut layer's mark is placed once on the original uncut carrier, while only its painted path is split; this keeps the mark present without duplicating it on every fragment. A cut layer cannot participate in a subgraph underlay. Self, unknown, and invisible references are reported as errors. A valid edge pair with no proper interior intersection, including one that only shares an endpoint, is left unchanged.

```
#let crossed-style(edge) = if edge.eid == 7 {
  (stroke: black, pattern: "coil", crossing-under: 5, crossing-gap: 0.55)
} else {
  (stroke: black)
}
```

For dangling edges, `edge-dangling-tangent: "horizontal"` or `"vertical"` constrains the tangent at the free edge position while retaining the edge's `bend`; the per-layer override is `dangling-tangent`. `auto` keeps the ordinary bent route, and `anchor-control-distance` controls the constrained handle length. `bend` affects dangling edges only. A paired edge instead follows its source node, `edge.pos`, and sink node, with `edge-omega` controlling its Hobby curve.

For anchored paired edges, `anchor-control-distance` sets the control-handle distance in graph units, not the endpoint position. Each edge can specify `source-style: (anchor-control-distance: 4)` and `sink-style: (anchor-control-distance: 1.5)` to tune the two ends independently. Set `source-anchor` and `sink-anchor` in its `edge-style` to choose the tangent directions. A shared `edge-style: (anchor-control-distance: 2)` supplies both ends unless an endpoint style overrides it; `auto` is computed independently for each end. An override no longer supplies the opposite endpoint's distance.

```
#draw(
  g,
  edge-split-gap: 0.28,
  edge-dangling-tangent: "horizontal",
  source-style: edge => (stroke: black, split-gap: 0.4),
  sink-style: edge => (stroke: black, split-gap: 0.4),
)
```

Data defaults are also the global styling hook for all sources, sinks, nodes, and edges. More specific data can be added with captured named arguments on node, edge, source, and sink items, or by running `graph.map`/`graph.eval-fields` after construction. This keeps evaluated Typst values in the native data channel instead of adding renderer-specific eval fields to the Rust topology spec:

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(
    source(<a>),
    <a-c>,
```

```
    sink(<c>),
    label: [a-c],
    kind: "highlight",
  )
},
  name: "demo",
  default-source-data: (style: (stroke: red + 0.5pt)),
  default-sink-data: (style: (stroke: blue + 0.5pt)),
)
#let endpoint-style(edge, side) = {
  let endpoint = edge.at(side + "-half-edge", default: none)
  if endpoint == none { return (:) }
  endpoint.data.at("style", default: (:))
}
#draw(
  layout(g),
  source-style: edge => endpoint-style(edge, "source"),
  sink-style: edge => endpoint-style(edge, "sink"),
)
```

`graph.build` also accepts comma-separated items:

```
#let g = build(
  node(<a>),
  node(<b>),
  edge(
    source(<a>, compass: "e"),
    <ab>,
    sink(<b>, compass: "w"),
    label: [ab],
    statements: (color: "0055ff", label: "ab"),
  ),
  name: "demo",
  statements: (full_num: "x + y"),
  default-node-statements: (shape: "circle"),
  default-edge-statements: (
    color: "000000",
    display-label: "$#label$",
  ),
)
```

Graph Queries

`graph.info(g)` returns graph metadata. `nodes(g)` returns node records, and `edges(g)` returns edge records. Node and edge record `name` values are Typst labels when present. Pass `subgraph: sg` to filter nodes or edges by a compatible subgraph object. Nodes must be incident to a selected half-edge; an edge is included if either half is selected, but its record keeps both available endpoints. These queries do not extract or open topology.

After `graph.cut`, node, edge, and half-edge records carry `origin` relative to the immediate input graph. Boundary nodes and dangling cut edges also carry `boundary`. Use `graph.boundaries(view)` for a uniform endpoint query, including current positions after placement or layout. Auxiliary nodes remain visible in `graph.nodes(view)`; filter with `node.boundary == none` to exclude them.

`graph.join(left, right, key: "statement")` joins dangling half edges with opposite flows. The supported keys are `"statement"`, `"compass"`, `"port-label"`, and `"id"`; unmatched half edges remain in partial joins. Matching identities must be unique within each input. Node and edge labels that occur in both inputs are rejected. Graph, node, edge, and endpoint data is preserved: right-only dictionary fields are retained and conflicting fields use the left value. The left graph supplies the name and layout configuration. Use `graph.join-edges` when the matching key is an edge-level statement rather than half-edge metadata.

`graph.cycles(g)` returns subgraph objects for a cycle basis. `graph.forests(g)` returns subgraph objects for spanning forests. Both return arrays of the same topology-bound dictionaries accepted by `subgraph`, graph queries, `layout`, and `draw`, not arrays of raw selection bytes.

Default diagram layout

The shared renderer used by Python graph rendering, Feynman diagrams and `just draw` defaults to `"impred"`. Direct Typst can request the same pipeline:

```
#let g = layout(g,
  layout-algo: "impred",
  impred-parallel-balance: 1,
  impred-pull: 0.45,
)
#draw(g)
```

This runs EC-constrained planarization, then constrained ImPrEd relaxation. Both stages execute inside the bundled `linnest.wasm` plugin. No Python process or external layout service is needed. The seed keeps incoming and outgoing exterior groups consecutive; individually optimal edge insertions introduce necessary crossings. Their sequence is a heuristic for the complete graph's crossing number. All-incoming and all-outgoing diagrams use radial placement.

Relaxation preserves the carrier's crossing configuration while respecting native pins and coordinate groups. Topology-normalized outward pull can lengthen external legs, and long external segments may gain route points. Parallel internal edges receive attraction divided by their multiplicity raised to `impred-parallel-balance`; its default `1` enlarges self-energy bundles relative to bridge edges. `0` disables that correction.

The final drawing uses Hobby through each complete physical route, then the usual particle styling and joint momentum-arrow/label placement. Graph springs and graph annealing are not additional default passes. Hobby smoothing and decorative coils do not themselves carry the polyline crossing certificate.

ImPrEd relaxes no labels itself, so in every `label-layout` its internal particle labels follow their carriers: the shared label annealing step slides each along its edge and chooses a side. Their clearance is at least the edge's `label-gap` (default `0.15`) and clears a coil or wave band, `0.06` plus its amplitude. The `label-layout` relaxation models apply only to the force and anneal layouts.

`impred-steps` sets the nominal cooling budget. The first 20% uses fine integration steps; afterward `impred-step-scale` permits a larger stride, scaling both the force displacement and movement cap. Strides stop at the original 25-tick geometry/refinement checkpoints and 100-tick progress checkpoints. Every movement still passes the native constraint certificate. The native report's `iterations` counts actual solves; progress callbacks use nominal ticks. `impred-step-scale: 1` retains the reference schedule, while the default `2` reduces work with small numerical layout differences.

Distance-based forces cannot turn a drawing, and the weak outward pull turns it only slowly; a long schedule mostly waits for that rotation. With mixed incoming and outgoing legs, `impred-level` therefore rotates the whole drawing about its vertex centroid to the exact external-pull optimum at every refinement checkpoint, so the legs settle horizontally and the default budget is `500` steps. A rotation is halved until directed coordinates and crossing witnesses remain valid. Pinned or aligned coordinates define the frame and skip levelling; `impred-level: false` restores the unrotated schedule.

Option	Default	Meaning
`impred-spacing`	`2.4`	Characteristic force and movement length.
`impred-repulsion`	`2.5`	Point repulsion multiplier.
`impred-attraction`	`2.5`	Route-segment attraction multiplier.
`impred-parallel-balance`	`1`	Parallel-edge attraction exponent.
`impred-pull`	`0.45`	Base outward force on external endpoints.

`impred-pull-balance`	`1`	Topology-normalization exponent.
`impred-pull-attachment`	`4`	Extra demand from distributed attachments.
`impred-external-max-points`	`2`	Maximum extra external route points, from 0 to 3.
`impred-split-length-ratio`	`1.5`	External subdivision threshold relative to spacing.
`impred-contract-chord-ratio`	`1.25`	External contraction threshold; must be below the split threshold.
`impred-edge-clearance`	`0.4`	Node-to-edge clearance scale.
`impred-node-edge-strength`	`4`	Node-to-edge force multiplier.
`impred-steps`	`500`	Nominal cooling budget.
`impred-step-scale`	`2`	Positive maximum integration stride; `1` uses the reference schedule.
`impred-level`	`true`	Rotate mixed-flow drawings to the external-pull optimum at checkpoints.

The same names work through CLI input overrides:

```
just draw --input impred-parallel-balance=1 --input impred-pull=0.45 --input impred-step-scale=1
```

Explicit `"force"`, `"anneal"`, `"tree"`, `"dot"`, and `"stable-layered"` layouts remain available. The low-level `layout(g)` function retains its force default; the shared drawing renderer selects ImPrEd explicitly. ImPrEd currently lays out the complete graph. For a partial layout pass using `layout(..., subgraph: ...)`, select one of the existing algorithms below. Drawing or highlighting a subgraph still uses the complete graph layout.

Spring and annealing layout model

The `"force"` and `"anneal"` algorithms start from a traversal-tree placement, then optimizes the positions of graph nodes and edge control points. The initial tree spacing is

$$L = \lambda \sqrt{\frac{WH}{\max(n, 1)}}$$

,

with horizontal spacing $\tau_x L$ and vertical spacing $\tau_y L$. Here λ is `length-scale`, W is `viewport-w`, H is `viewport-h`, τ_x is `tree-dx`, and τ_y is `tree-dy`. These fields set the geometry scale for both layout modes. Isolated subgraph layout uses the number of selected incident nodes for n; other layouts use the full graph node count.

For deterministic, non-iterative placement, use `layout-algo: "tree"` or `layout-algo: "dot"`. `"tree"` places a traversal forest by levels. `"dot"` uses a directed layered placement for acyclic inputs, falling back to tree placement when the selected edges contain a cycle. Both modes also work on a subgraph:

```
#let g = parse("digraph partial { a -> b; b -> c; c -> d; d -> a }").at(0)
#let tree = graph.forests(g).at(0)
#let g = layout(g, layout-algo: "tree", subgraph: tree)
#draw(g)
```

Only nodes touched by the selected subgraph are placed. Edge control points are then resolved from the current node positions, so edges outside the selected subgraph are drawn as straight lines unless their own position constraints say otherwise.

`layout-algo: "force"` and `layout-algo: "anneal"` also accept `subgraph`. The default `solver.subgraph-mode: "fixed-boundary"` keeps nodes and edge control points outside the selection fixed while retaining their spring, repulsion, centroid, and crossing interactions. Set `solver: (subgraph-mode: "isolated")` to omit unselected half edges, nodes, control points, and labels from those interactions. Selected halves of otherwise paired edges become dangling boundaries of the isolated solver domain. Complement

coordinates and labels remain unchanged, and an empty selection is a no-op. An isolated selection cannot split a grouped coordinate; expand the selection or remove that group.

Set `layout-nodes: "fixed"` to keep every node at its current `pos` for this layout pass and move only edge control points. With `subgraph`, only edges in the selected subgraph are moved; all other edge control points keep their current positions. The fixed-node policy is temporary; the returned graph stores the resulting coordinates as `pos`, but it does not turn them into persistent `pin` constraints. This is useful after one node-placement pass when a later pass should route or relax selected edges without disturbing the node layout:

```
#let g = parse("digraph partial { a [pos=\"0,0\"]; b [pos=\"4,0\"]; c [pos=\"8,0\"]; a -> b; b ->
c }").at(0)
#let first = subgraph.bits(g, (true, true, false, false))
#let g = layout(g, layout-algo: "tree", layout-nodes: "fixed", subgraph: first)
#draw(g)
```

Related controls can be supplied as shallow semantic records. These records only name graph-layout concepts; they do not select a drawing domain or style:

```
#let common = layouts.options(
  spring: (strength: 8, length: 0.5, rest-length-scale: 0.75),
  repulsion: (edge-node: 0.05, dangling: 2),
  constraints: (side-strength: 5, external-pull: 1, external-pull-balance: 1),
  labels: (steps: 0),
  solver: (
    algorithm: "force", steps: 50,
    initial-repulsion: 0.15, repulsion-growth: 0.7,
  ),
)
#let spacious = layouts.options(
  base: common,
  spring: (length: 0.7),
  repulsion: (dangling-centroid: 3),
)
#let g = layout(g, ..spacious)
```

`layouts.options` performs a sparse update: the second spring record above changes `length` without losing the inherited `strength`. Grouped fields override their corresponding flat arguments. Flat arguments remain useful for numerical experiments and controls that do not have a semantic grouping.

An edge may carry any number of intermediate route points, independently of other edges. Set `route-points` on `graph.source(...)` or `graph.sink(...)`, or on a half-edge through `graph.map`. Each list runs from its incident node toward the edge's central layout point, excluding those two endpoints. In Python, use `half_edge.drawing.route_points = [(x, y), ...]`. Coordinates use the same layout units as node and edge positions.

Force and annealing layouts retain these points as movable coordinates. Springs measure the complete half-route length; adding collinear points does not change the spring's preferred length or stiffness. The edge's repulsion charge is shared equally between its central point and active route points, so extra bends do not multiply its total charge. This retains the existing local edge–edge, edge–vertex and vertex–vertex interactions. Fixed edge axes freeze the corresponding bend coordinates; unselected half-routes stay fixed. Grouped placement constraints still refer to the central edge point. Routed annealing evaluates the full energy after proposals.

A full layout records its solved geometry. Subsequent force or annealing passes start from those node positions, central edge points and half-routes; they do not restart the traversal-tree initializer. The representation is shared by layered layouts, manual geometry and solver refinement. Route point counts may differ between edges, but stay fixed during each solver run.

Rendering fits a Hobby curve through the resulting points. Empty route lists retain the usual single-point layout and curve construction. An explicitly requested force pass may refine an ImPrEd result, but does

not retain the ImPrEd carrier's crossing certificate. A native `ForceLayoutStream` can start from a routed `TypstGraph`; its frames include complete source-to-sink `paths`, as well as node and central edge positions.

`spring-length-scale` (also `spring.rest-length-scale`) multiplies preferred incidence spring lengths independently of repulsive strengths. It must be positive and finite, defaults to `1`, and combines multiplicatively with each edge's `spring-length`. This changes the graph's geometry rather than zooming its rendering. In contrast, `length-scale` (also `spring.length`) sets the common length scale L, which also determines charges and numerical scales. Use the drawing `unit` to resize a finished diagram without changing layout.

The shared spring/charge model uses the following coefficients:

$$c_{\mathrm{vv}} = \beta L^3$$

$$c_{\mathrm{ev}} = \beta \gamma_{\mathrm{ev}} L^3$$

$$c_{\mathrm{ee}} = \beta \gamma_{\mathrm{ee}} L^3$$

$$c_{\mathrm{center}} = \beta g_{\mathrm{center}}$$

$$c_{\mathrm{dangling}} = \beta \gamma_{\mathrm{dangling}} L^3$$

$$c_{\mathrm{dangling\text{-}centroid}} = \beta \gamma_{\mathrm{dangling\text{-}centroid}} L^3$$

The Typst parameter names are `beta` for β, `gamma-ev` for γ_{ev}, `gamma-ee` for γ_{ee}, `g-center` for g_{center}, `gamma-dangling` for $\gamma_{\mathrm{dangling}}$, and `gamma-dangling-centroid` for $\gamma_{\mathrm{dangling\text{-}centroid}}$. The spring stiffness k is `k-spring`, and the softening constant ε is `eps`.

`external-pull` (also `constraints.external-pull`) sets a base force $F_0 = kLa$, where a is this parameter. When both momentum directions occur, incoming endpoints pull left and outgoing endpoints pull right. There is no finite target position: a longer chain of interactions can stretch farther. If every external momentum is incoming, or every one is outgoing, the force instead points radially away from the node centroid. This display rule uses momentum flow, independently of particle or fermion-arrow orientation.

Connected components containing both directions normalize their pull by the load carried through their springs. A unit total pull on each side is divided equally among that side's external legs. A linear spring-network solve then estimates the internal loads, accounting for parallel routes and cancellation where incoming and outgoing legs share a node. The reciprocal of the largest load, including the dangling springs, gives a width W. Each endpoint uses the baseline multiplier $\frac{W}{N_{\mathrm{side}}}$, where N_{side} counts the component's external legs with that endpoint's direction. Components with only one external direction keep $w_e = 1$.

Endpoints constrained to move together in X can attach at different places along the internal network. Their common external line must clear the furthest attachment, rather than the average attachment. Let u_v be the unit-load spring-network displacement at an attachment node, and let s be -1 for incoming legs and $+1$ for outgoing legs. Within each movable X group, connected component, and direction, the multiplier is $w_e = \frac{W}{N_{\mathrm{side}}} + cW\,\mathrm{mean}_i\left(\max_{j(su_j)} - su_i\right)$. The mean counts external legs, including repeated attachments at a node. Independent endpoints and groups whose attachments have the same displacement retain the baseline. Fixed X coordinates receive no group correction. This uses connectivity and coordinate constraints, without measuring the current drawing or adding a target position. The correction can exceed one, and opposite sides need not receive equal total pull.

`external-pull-attachment` (also `constraints.external-pull-attachment`) controls c, the strength of this distributed-attachment correction. Its default is `1`. Set it to `0` to retain only bottleneck balancing, or above `1` to increase pull where a shared external line spans distinct network displacements. It accepts finite non-negative values. Unlike increasing the balancing exponent, increasing this setting never decreases the corrected multiplier, even when it is below one. Terminal groups, independent endpoints, fixed X coordinates and one-direction radial pull receive no extra force.

A series chain with one external leg at each end keeps its original pull regardless of its length. Parallel routes share the internal load, while dangling springs still limit the normalization. For example, two incoming and two outgoing legs separated by one internal bridge each receive $\frac{F_0}{2}$; three parallel internal edges with one external leg on each side keep F_0 on those legs. Distributed attachments sharing an external line can need more pull than these terminal attachments. This linear estimate does not guarantee equal strain or outward endpoints in the nonlinear layout with repulsion, curvature, and pins.

`external-pull-balance` (also `constraints.external-pull-balance`) controls the strength of the topology adjustment. Its default is `1`, retaining the balancing above. Set it to `0` for uniform pull per external endpoint, a value between `0` and `1` for weaker balancing, or a value above `1` for stronger balancing. It accepts finite non-negative values; combinations whose effective force overflows are rejected. For example, `2` gives the bridge example above one quarter of the base force per leg; the Higgs sunset with one leg on each side keeps its base force. This setting does not change one-direction radial pull or a series chain with one external leg at each end. Values above `1` decrease multipliers below one further and increase multipliers above one further. The reaction on the nodes prevents translational drift.

Writing this balance as b, the endpoint force is $F_e = F_0 w_e^b$. Using an exponent keeps the pull non-negative. Its contribution to energy is $-\sum_e F_e s_e (x_e - \overline{x})$ in the mixed case, where s_e is -1 for incoming and $+1$ for outgoing; in the radial case it is $-\sum_e F_e d(e, \overline{v})$. The weights depend on active topology and coordinate groups, so moving points or changing per-edge rest lengths does not change them. The opposite reaction is shared over the nodes; pins are respected in both force and anneal modes. The generic `external-pull` default is zero (disabled). The shared physics template uses `external-pull: 1` and `spring-length-scale: 0.75`.

In `layout-algo: "anneal"`, linnest minimizes an energy:

$$E = \sum_{i<j} \frac{1}{2} \frac{c_{\text{vv}}}{d(v_i, v_j) + \varepsilon} + \sum_{i,e} \frac{c_{\text{ev}}}{d(v_i, e) + \varepsilon} + \sum_{(v,e)\ \text{incident}} \frac{1}{2} k (\ell_e - d(v, e))^2$$
$$+ \sum_{\text{local edge pairs}} \frac{1}{2} \frac{c_{\text{ee}}}{d(e_i, e_j) + \varepsilon} + \sum_{\text{dangling pairs}} \frac{1}{2} \frac{c_{\text{dangling}}}{d(e_i, e_j) + \varepsilon} + \sum_{\text{dangling } e} \frac{c_{\text{dangling-centroid}}}{d(e, \overline{v}) + \varepsilon} + \sum_i \frac{1}{2} c_{\text{center}} d(v_i, 0)^2 + E_{\text{pull}} + p_{\text{cross}} N_{\text{cross}}$$

.

Here $\overline{v}$ is the mean node position. The centroid term pushes every dangling endpoint away from that mean; its equal-and-opposite reaction is shared over the nodes so it introduces no net force. p_{cross} is `crossing-penalty` and N_{cross} is the number of detected edge crossings. E_{pull} is the constant external-pull contribution defined above. The quadratic center term pulls nodes toward the origin at every radius. `temp`, `step`, `seed`, `steps`, `epochs`, `cool`, `accept-floor`, `step-shrink`, and `incremental-energy` belong to this simulated annealing mode. `crossing-penalty` also contributes to the discrete reordering steps in force mode, described below.

In `layout-algo: "force"`, linnest applies the direct forces corresponding to the same vertex-vertex, edge-vertex, incidence spring, local edge-edge, dangling-edge, dangling-centroid, constant external-pull, and center terms. `step` is the integration step, `delta` clamps per-step movement, and `steps` and `epochs` set the iteration budget. `cool` shrinks the step after each epoch. `early-tol` can stop the run when movement is small only after effective depth reaches zero and repulsion reaches its final strength; small movement or a cooled-to-zero step cannot skip either schedule.

During planar relaxation, force layout can exchange the free coordinates of adjacent points on the same constrained line. This allows a crossed ordering to change without forcing repelling points through one another. Nodes and edge control points use the same rule: their common coordinate must be fixed or grouped, and the coordinate being exchanged must be free. Fully pinned points, groups on the exchanged axis, and inactive points cannot be swapped. A swap is accepted only when it lowers the current planar layout energy, including `crossing-penalty`. The solver tries these moves at epoch boundaries and before stopping, then continues the same force run. There is no continuous crossing force, and nonplanar graphs may retain crossings.

The force-only `initial-repulsion` and `repulsion-growth` controls produce a single compact-to-relaxed run, without restarting or cycling. Both are finite fractions in `0.0=1` and can also be set in the `solver` record. For normalized progress `u` across the `repulsion-growth` fraction of the iteration budget, the multiplier is `initial-repulsion + (1 - initial-repulsion) * u^2 * (3 - 2*u)`. It grows to exactly one, and the remaining iterations settle with the final forces. It scales vertex-vertex, edge-vertex, local edge-edge, dangling-pair, and dangling-centroid repulsion; springs, constant pull, and center gravity retain their requested strengths. Cooling independently reduces the step size.

Generic defaults are `initial-repulsion: 1` and `repulsion-growth: 0.7`; setting the initial fraction to one or the growth fraction to zero starts at full strength. The physics template starts at `0.15` and reaches full repulsion after `0.7` of the budget. A growth fraction of one still evaluates the last iteration with full repulsion. These controls do not affect anneal mode. The one-way change from attraction-dominated organization to stronger repulsion is inspired by Toosi and Nikolov's Sync-and-Burst (2015); this solver retains its own force laws, integration, and cooling, rather than implementing their complete algorithm.

The force-only `depth-scale` (default `1.0`) and `flattening-end` (default `0.5`) control auxiliary depth. `depth-scale` must be finite and non-negative; `flattening-end` must be finite and in `0.0=1`. They can be supplied as flat arguments or in `solver: (depth-scale: 1.0, flattening-end: 0.5)`; grouped values override flat ones. The old depth-spring controls are not aliases.

Effective depth is `scale * raw-z`. The integrator starts with auxiliary raw depths to break overlaps and smoothly reduces `scale` from `depth-scale` to zero over the initial `flattening-end` fraction of the total `steps` × `epochs` iteration budget. For normalized progress `u` from 0 to 1 over that interval, the smoothstep scale is `depth-scale * (1 - u)^2 * (1 + 2 * u)`. The remaining iterations relax projected overlaps with effective depth exactly zero, so repulsion cannot be satisfied by invisible separation. `flattening-end: 0` or `depth-scale: 0` starts in 2D; `flattening-end: 1` still evaluates the final iteration on the exact plane.

There is a single original iteration budget and cooling schedule: flattening neither adds a second phase nor restarts `step`. Unpinned raw depths stay bounded by the initial spread and supplied depth magnitudes; hard raw-depth pins remain fixed even after their effective depth has flattened to zero. XY pins, shared coordinates, and fixed subgraph boundaries apply throughout; label layout runs only after this single graph pass. Output positions remain 2D, and auxiliary z does not alter draw order. Flattening is exact, while force convergence remains limited by the iteration budget and movement tolerance.

`directional-force` is applied in both modes as an extra bias derived from pin/port direction constraints.

After either graph layout mode, labels are relaxed separately. If L_l is the label target distance and q_l is the label repulsion strength, then $L_l = \alpha_l L$ and $q_l = \beta_l L^2$. The Typst names are `internal-label-length-scale` for α_l on paired edges, `external-label-length-scale` on dangling edges, and `label-charge` for β_l. The semantic `labels` dictionary exposes these as `internal-distance` and `external-distance`; changing either leaves the other target distance unchanged. `label-spring` is the spring constant pulling each label toward its target. `label-layout: "normal"` uses a perpendicular offset target. With `label-layout: "dangling-tangent"`, paired edges still use that perpendicular target, but dangling half-edge labels are offset along the edge direction away from the attached node. With `label-layout: "fixed-length"`, relaxation keeps the label at distance L_l from the edge point and only rotates it under repulsive forces. `label-steps`, `label-step`, `label-early-tol`, and `label-max-delta-scale` control the label relaxation iteration.

`graph.style` measures node and label sizes before layout; those sizes enter repulsion during label relaxation. For labels with measured width and height, a subsequent box-collision pass starts from the relaxed positions. It keeps each position when its box clears node boxes and previously placed label boxes, otherwise searching nearby candidates. This correction may change the distance from the edge point, including in `"fixed-length"` mode.

Repulsion considers edge layout points, and box correction considers nodes and labels; neither guarantees clearance from complete edge curves or drawing layers such as momentum arrows. Attached path labels may also be positioned again during drawing, as described above.

Subgraphs

A subgraph is a dictionary containing half-edge membership, a topology signature, subgraph-wide `data`, and per-half-edge `hedge-data`. Annotations remain Typst values: content and callbacks are not serialized through Wasm. Independent selections can annotate the same hedge without changing its graph data or each other.

- `subgraph.select(g, nodes: (), edges: (), source: (), sink: (), hedges: ())` selects the union of the supplied references. Node and edge references are Typst labels or numeric IDs, supplied in arrays. `nodes` selects incident hedges, `edges` selects both available halves, and `source` / `sink` selects the named edges' structural halves, independently of drawing orientation. `hedges` selects exact numeric hedge IDs. Unknown references and missing requested halves are errors; an isolated node contributes no hedges.
- `subgraph.label(g, label)` constructs a subgraph from a base62 label.
- `subgraph.bits(g, bits)` constructs a subgraph from a boolean hedge array.
- `subgraph.compass(g, compass)` selects half edges with a DOT compass point.
- `subgraph.hedges(sg)`, `subgraph.contains(sg, hedge)`, and `subgraph.contains-edge(sg, edge)` inspect membership. The last checks whether either half of an edge record is selected.
- `subgraph.complement(g, sg)` selects all other hedges. It preserves subgraph-wide data, but the newly selected hedges have no annotations.

`subgraph.with-data(g, sg, data: value, hedge: updater)` returns an annotated selection. `data: auto` preserves subgraph-wide data and `hedge: none` preserves hedge annotations. A non-function `hedge` value replaces every selected hedge's annotation. A callback receives its original endpoint record plus `edge`, `edge-name`, and `flow`; `data` holds the existing selection annotation and `graph-data` holds the graph's half-edge data. Its return value replaces the annotation, even when it is `none`. To store a callback as an annotation, return it from the updater rather than passing it as the updater itself. `subgraph.hedge-data(sg, h)` reads one selected hedge's annotation.

Graph-aware consumers check a signature of IDs, names, and source/sink incidence, not graph identity or placement. Selections survive layout and drawing/data changes, but incompatible topology or names are rejected. Re-select on a cut view rather than reusing its master's selections.

Breaking change: raw subgraph bytes are no longer accepted by public APIs. Use the constructors above, or the wrapped results of `graph.cycles` and `graph.forests`, and pass the whole object, not its internal `bytes` field. `subgraph.to-label(sg)` exports membership only: rebuilding with `subgraph.label(g, label)` binds it to that graph but does not restore annotations or the old topology signature. Keep the object to preserve Typst sidecar data; a base62 label is not a complete serialization of an annotated selection.

Passing `subgraph: sg` to `draw` highlights half-edges, while `layout` restricts placement or optimization. Neither opens edges; use `graph.cut` for that.

Directed Cut Views

`graph.cut(g, left: left, right: right, boundary: patch)` opens **one weighted directed cut** and leaves `g` unchanged. The two selections are its drawing sides, not separate cuts or a partition of the vertices. They must be disjoint and contain exactly opposite halves of every selected paired edge in `g`. Already dangling edges cannot be selected for opening.

Think of the drawing boundaries as L and R, with a positive seam passage from R to L. Selecting an edge's source on R and sink on L follows that positive direction; swapping them reverses it. This is independent of superficial drawing orientation, including reversed fermion arrows. Side names do not place endpoints automatically: `boundary` supplies the view's geometry.

An annotation `(winding: n)` on either selected half requests a positive integer number of same-direction passages; it defaults to one. If both halves specify winding, they must agree. Each selected edge becomes a source stub, a sink stub, and `n - 1` middle segments. Thus winding two produces three segments without separate named seams or a caller-supplied ordering of cut objects. Winding is not a net count of cancelling back-and-forth crossings.

This small example uses the manual's existing imports. It stores content on the original edge and endpoints, annotates the cut sides, and opens one edge with winding two. The boundary callback places both ordinary free endpoints and the generated middle endpoints using the same record interface:

```
#let master = graph.build({
  node(<a>, pos: graph.pos(x: 0, y: 0))
  node(<b>, pos: graph.pos(x: 0, y: -2))
  edge(
    <link>,
    source(<a>, caption: [output]),
    sink(<b>, caption: [input]),
    label: [connection],
  )
})
#let left = subgraph.with-data(
  master, subgraph.select(master, sink: (<link>,)),
  data: [left boundary],
  hedge: h => (winding: 2, caption: h.graph-data.caption),
)
#let right = subgraph.with-data(
  master, subgraph.select(master, source: (<link>,)),
  data: [right boundary],
  hedge: h => (caption: h.graph-data.caption),
)
#let view = graph.cut(master, left: left, right: right, boundary: record => {
  let b = record.boundary
  (pos: graph.pos(
    x: if b.side == "left" { -3 } else { 3 },
    y: -2 * b.crossing,
  ))
})
#let view = layout(view, layout-algo: "tree")
#assert(graph.edges(master).len() == 1)
#assert(graph.edges(view).len() == 3)
#assert(graph.edge-data(view, <link.1>).label == [connection])
#assert(graph.boundaries(view).len() == 4)
#draw(view, edge-label: edge => edge.label)
```

Fragments are named `<link.0>` through `<link.n>` in underlying source-to-sink order, irrespective of L/R or drawing arrows. Unnamed edges use the native prefix `__linnest_cut_edge_ID`; generated name collisions are errors. Prefer explicit edge names when patching a view. Node, edge, and hedge `origin` refer to the immediate input graph, including on repeated cuts. Edge origins contain the input `edge` ID and `name`, plus `segment` and `winding`; uncut edges have `segment: none` and `winding: 0`. Node and hedge origins are input IDs; newly generated nodes have `origin: none`.

Physical graph, node, edge, and half-edge Typst data are remapped, including content and callbacks. Middle source hedges inherit the original sink hedge's data, and middle sink hedges inherit the original source's data, following the boundary side they continue. The cut resets fragment geometry for re-layout, rather than copying an old midpoint pin to new endpoints. Apply view-specific bends, placement, labels, and `crossing-under` targets with `graph.map`; a reference to a split edge needs the appropriate fragment name.

The `boundary` callback runs only on newly created endpoints, receiving a node or dangling-edge record and returning ordinary `graph.map`-style placement/data patches. Its `record.boundary` has output `edge` and optional `node` IDs, the original side's `hedge`, `side` (`"left"` or `"right"`), zero-based `crossing` along underlying flow, `data` from that hedge's annotation, `cut-data` from that side's subgraph-wide data, and the fragment's `origin` at creation. For example, the code above preserves each endpoint caption in `boundary.data.caption`.

On subsequent cuts, `boundary.origin` and `boundary.hedge` still refer to the input to the cut that created the endpoint; `side`, `crossing`, `data`, and `cut-data` are retained. Only `boundary.edge` and `boundary.node` remap to

current anchors. Inherited boundaries retain their placements and annotations without invoking the callback again.

`graph.boundaries(view)` returns these descriptors plus current `pos`, not coordinates cached at cut time. A dangling endpoint uses its edge position and has `node: none`; a middle endpoint uses its auxiliary node position. Query after layout, or inside `draw-after`, to group solved endpoints into background boxes with existing CeTZ primitives. Auxiliary boundary nodes are automatically zero-sized and unpainted, including their labels and custom node-drawing callbacks; they remain addressable anchors for incident edges.

Generated Reference

graph

- from-spec()
- build()
- parse()
- node()
- source()
- sink()
- edge()
- group()
- pin()
- start()
- pos()
- map()
- style()
- eval-fields()
- info()
- dot()
- nodes()
- edges()
- node-data()
- edge-data()
- update-node-data()
- update-edge-data()
- cut()
- boundaries()
- join()
- join-edges()
- cycles()
- forests()

from-spec

Construct a graph from a versioned Linnest graph-spec CBOR document.

This is the native renderer boundary. Unlike `parse`, it does not parse DOT.

Parameters

```
from-spec(input) -> dictionary
```

build

Build one graph object from a stream of node and edge items.

Use `node`, `source`, `sink`, and `edge` to create graph items. Positional items may be passed as comma-separated arguments or yielded from a Typst code block.

Parameters

```
build(
  ..items: array,
  name: none string,
  data: any,
  statements: dictionary,
  default-node-statements: dictionary,
  default-edge-statements: dictionary,
  default-node-data: any,
  default-edge-data: any,
  default-source-data: any,
  default-sink-data: any
) -> dictionary
```

..items `array`

Node and edge items returned by `node` and `edge`. The best way to use these is to use a code scope, so that the edges and nodes append each other:

```
#let g = build({
  node(<a>, label: [a])
  node(<b>, label: [b])
  node(<c>, label: [c])
  edge(source(<a>), sink(<b>))
  edge(source(<b>), sink(<c>))
  edge(source(<a>), sink(<c>))
})
```

name `none` or `string`

Graph name.

```
#let g = build(name: "My Graph", {
})
#info(g).name
```

Default: `none`

data `any`

Native Typst graph data.

```
#let g = build(data: (a:(b:(1, ))), {
})
#info(g).data
```

Default: `none`

statements `dictionary`

Flat graph statements, that get turned into a string to string dictionary in rust. Used by DOT parsing; values cannot nest.

```
#let g = build(statements: (a:1), {
})
#info(g).global-statements
```

Default: `(:)`

default-node-statements `dictionary`

Flat default node statements. Used by DOT; values cannot nest.

Default: `(:)`

default-edge-statements `dictionary`

Flat default edge statements. Used by DOT parsing; values cannot nest. These are applied/delegated to all edges.

```
#let g = build(default-edge-statements: (a:1), {
   node(<a>)
   edge(source(<a>))
   edge(sink(<a>))
})
#edges(g).map(e=>e.statements)
```

Default: `(:)`

default-node-data `any`

Default data merged into every node data. Captured node data fields override it.

```
#let g = build(default-node-data: (a:1), {
   node(<a>)
   node(<b>,a:2)
   node(<c>,b:(a:1))
})
#nodes(g).map(n=>n.data)
```

Default: `none`

default-edge-data `any`

Default data merged into every edge data. Captured edge data fields override it.

```
#let g = build(default-edge-data: (a:1,b:[$m_mu$]), {
   node(<a>,id:0)
   edge(source(<a>),id:1)
   edge(sink(<a>),<e>,id:0,b:[#set text(font:"Reforma")
   This is a test: ])
})
#edges(g).map(e=>e.data.b).join()
```

Default: `none`

default-source-data `any`

Default data merged into every source half-edge data. Captured source data fields override it.

```
#let g = build(default-source-data: (a:1), {
   node(<a>)
   edge(source(<a>))
   edge(sink(<a>))
})
#edges(g).map(e=>if e.source != none {e.source.data} else {none})
```

Default: `none`

default-sink-data `any`

Default data merged into every sink half-edge data. Captured sink data fields override it.

```
#let g = build(default-sink-data: (a:1), {
   node(<a>)
   edge(source(<a>))
   edge(sink(<a>))
})
#edges(g).map(e=>if e.sink != none {e.sink.data} else {none})
```

Default: `none`

parse

Parse one or more DOT graphs into graph objects.

Default data are applied before `eval-*` fields are evaluated, so default data strings can refer to parsed record fields such as `#str(name)`. Parsed fields take precedence over default data fields, so DOT `label="..."` overrides `default-node-data: (label: ...)`.

Linnest reads DOT syntax with Linnet's DOT reader and then gives special meaning to a small set of attributes. Every other attribute is preserved as a flat string statement. Preserved statements are visible through `info`, `nodes`, `edges`, `map`, and `eval-fields`, but they do not become native Typst `data` unless a matching `eval-*` argument is passed.

````
#let src = ```dot
digraph { a [label="A"]; }
```
#let n = nodes(parse(src.text).first()).first()
#n.statements.label
#n.data
````

Graph-level handling:

- The DOT graph name becomes `graph.info(g).name`. A graph-level `name` attribute can name an anonymous graph; a named `graph`/`digraph` header takes precedence.

````
#let src = ```dot
digraph header_wins { graph [name="ignored"]; a }
```
#info(parse(src.text).first()).name
````

- Top-level `key=value` statements and `graph [key=value]` attributes become `graph.info(g).global-statements`, except for the consumed `name`. They do not configure layout, even when a key looks like a layout option; pass layout options directly to `layout`.

```
#let src = ```dot
digraph { steps=1; graph [subtitle="nested"]; a }
```
#let statements = info(parse(src.text).first()).global-statements
#statements.steps
#statements.subtitle
```

- `node [key=value]` and `edge [key=value]` become `default-node-statements` and `default-edge-statements`. They are merged into each parsed node or edge before local attributes, so local attributes override defaults.

```
#let src = ```dot
digraph { node [color=gray]; edge [particle=g]; a [color=red]; a -> b }
```
#let g = parse(src.text).first()
#nodes(g).first().statements.color
#edges(g).first().statements.particle
```

Node handling:

- The DOT node id becomes the node `name` returned by `nodes`. If the DOT node id is numeric, it is consumed as the node index instead and the public `name` is `none`.

```
#let src = ```dot
digraph { alpha; 1; }
```
#nodes(parse(src.text).first()).map(n => n.name)
```

- `id=<n>` is also consumed as an explicit node index. Explicit node indexes must be unique and in bounds after parsing.

```
#let src = ```dot
digraph { a [id=1]; b [id=0]; }
```
#nodes(parse(src.text).first()).map(n => n.name)
```

- `style=invis` marks the DOT node as a dangling external endpoint. It is not returned by `nodes`. Its remaining attributes are copied onto the external edge that touches it, except `shape` and `label`; `style` and `id` are consumed.

```
#let src = ```dot
digraph { ext [style=invis, column=left, label=skip]; ext -> a [id=0]; }
```
#let g = parse(src.text).first()
#nodes(g).map(n => n.name)
#edges(g).first().statements.column
#edges(g).first().statements.at("label", default: none)
```

- Node `style` is consumed for every node; only `style=invis` has special behavior. Use another attribute name for drawing style metadata that must survive parsing.

```
#let src = ```dot
digraph { a [style=filled, "draw-style"=filled]; }
```
#nodes(parse(src.text).first()).first().statements
```

- `pos` is parsed as node placement. Simple numeric values are exposed as `node.pos`; extended placement values are used by `layout`. `pin` is an explicit layout constraint and is not exposed as a public statement. The `pos-x-set` and `pos-y-set` record fields say which axes were explicitly supplied, including for partial placements whose resolved `pos` contains both coordinates. `dot` preserves them as canonical Linnest DOT extension attributes.

```
#let src = ```dot
digraph { a [id=0, pos="0,0!"]; b [id=1, pos="ref(node:0)+2,0!"]; }
```
#let parsed-nodes = nodes(parse(src.text).first())
#parsed-nodes.map(n => n.pos)
#parsed-nodes.map(n => n.statements.at("pin", default: none))
```

- `shift` is parsed as a drawing shift and also remains available as a statement. `eval` is retained for legacy round-tripping. Other node attributes are preserved as node statements.

```
#let src = ```dot
digraph { a [shift="0.1,0", eval=legacy, label=A]; }
```
#let n = nodes(parse(src.text).first()).first()
#n.shift
#n.statements
```

Edge handling:

- `id=<n>` is consumed as the edge index. It is not preserved as a statement; drawing callbacks expose the resulting stable edge index as `eid`. Explicit edge indexes must be unique and in bounds. Without explicit ids, edge order is parser-internal, not DOT input order.

```
#let src = ```dot
digraph { a -> b [id=1, label=later]; b -> c [id=0, label=first]; }
```
#let parsed-edges = edges(parse(src.text).first())
#parsed-edges.map(e => e.edge)
#parsed-edges.map(e => e.statements.label)
#parsed-edges.map(e => e.statements.at("id", default: none))
```

- `dir=forward`, `dir=back`, and `dir=none` become edge orientations `"default"`, `"reversed"`, and `"undirected"`. If omitted, directed DOT edges are `"default"` and undirected DOT edges are `"undirected"`.

```
#let src = ```dot
digraph { a -> b [id=0]; b -> c [id=1, dir=back]; c -> d [id=2, dir=none]; }
```
#edges(parse(src.text).first()).map(e => e.orientation)
```

- `source="..."` and `sink="..."` are consumed as endpoint `statement` values and removed from edge statements. On an external edge, use the attribute matching the real endpoint side in the DOT edge: `ext -> a` uses `sink=...`, while `a -> ext` uses `source=...`.

```
#let src = ```dot
digraph { ext [style=invis]; ext -> a [id=0, sink=in]; a -> ext [id=1, source=out]; }
```
#let endpoint-statement(endpoint) = if endpoint == none { none } else
{ endpoint.at("statement", default: none) }
#let parsed-edges = edges(parse(src.text).first())
#parsed-edges.map(e => endpoint-statement(e.source))
#parsed-edges.map(e => endpoint-statement(e.sink))
#parsed-edges.map(e => e.statements.at("source", default: none))
```

- `pos` is parsed as the edge control-point placement. `pin` is an explicit edge layout constraint. `shift`, `label-pos`, `label-angle`, and `bend` are parsed into edge geometry fields; except for `pin`, they also remain available as statements. The `pos-x-set` and `pos-y-set` record fields identify explicitly supplied axes, including partial placements; `dot` preserves them as canonical Linnest DOT extension attributes. Other edge attributes are preserved as edge statements.

```
#let src = ```dot
digraph { a -> b [id=0, pos="0,1!", pin="x:@edge", shift="0.1,0", "label-pos"="0,1.2",
"label-angle"="0.3rad", bend="0.4rad", particle=g]; }
```
```

````
#let e = edges(parse(src.text).first()).first()
#e.pos
#e.shift
#e.label-pos
#e.label-angle
#e.bend
#e.statements.particle
#e.statements.at("pin", default: none)
````

- Attributes copied from an invisible endpoint node are merged into the external edge statements unless their keys are `shape` or `label`.

````
#let src = ```dot
digraph { ext [style=invis, column=left, shape=none, label=skip]; ext -> a [id=0]; }
```
#edges(parse(src.text).first()).first().statements
````

Port and half-edge handling:

- DOT ports of the form `node:port` and `node:port:compass` are preserved on source/sink endpoint records. Numeric ports also assign explicit half-edge ids. Non-numeric ports are exposed only as `port-label`.

````
#let src = ```dot
digraph { a:left:e -> b:0:w [id=0]; }
```
#let e = edges(parse(src.text).first()).first()
#e.source.port-label
#e.sink.hedge
````

- Compass values are exposed as `compass`; valid DOT compass names include `n`, `ne`, `e`, `se`, `s`, `sw`, `w`, `nw`, `c`, and `_`.

````
#let src = ```dot
digraph { a:n -> b:sw [id=0]; }
```
#let e = edges(parse(src.text).first()).first()
#e.source.compass
#e.sink.compass
````

- Explicit half-edge ids from numeric ports must be unique and in bounds. They are retained for half-edge ordering and for `join` with `key: "id"`; query and eval records expose the resulting half-edge index as `hedge`.

````
#let src = ```dot
digraph { a:1 -> b:0 [id=0]; }
```
#let e = edges(parse(src.text).first()).first()
#e.source.hedge
#e.sink.hedge
````

Placement fields:

- `pos="x,y"` gives a starting coordinate; `pos="x,y!"` pins both axes.

````
#let src = ```dot
digraph { a [id=0, pos="0,0"]; b [id=1, pos="2,0!"]; a -> b; }
```
#let parsed = parse(src.text).first()
#nodes(parsed).map(n => n.pos)
#nodes(layout(parsed, steps: 10, seed: 1)).map(n => n.pos)
````

- `pos="ref(node:<id>)+dx,dy!"` and `pos="ref(edge:<id>)+dx,dy!"` reference explicit DOT node or edge ids, not names and not implicit parse order.
````

```
#let src = ```dot
digraph { a [id=0, pos="1,1!"]; b [id=1, pos="ref(node:0)+2,0!"]; a -> b [id=0,
pos="ref(node:1)+0,1!"]; }
```
#let g = parse(src.text).first()
#nodes(g).at(1).pos
#edges(g).first().pos
```

- Axis form accepts `x:&lt;coord&gt;` and `y:&lt;coord&gt;` entries. `!` applies to the individual axis, for example `pos="x:2!,y:0"`.

```
#let src = ```dot
digraph { a [id=0, pos="x:2!,y:0"]; }
```
#nodes(parse(src.text).first()).first().pos
```

- Group coordinates use `@name`, `@+name`, or `@-name` in axis form and must be pinned with `!`, for example `pos="x:@-left!,y:@row!"`.

```
#let src = ```dot
digraph { ext [style=invis]; ext -> a [id=0, pos="x:@-left!,y:@row!"]; }
```
#edges(layout(parse(src.text).first(), layout-algo: "tree")).first().pos
```

- `pin` accepts numeric point constraints, `x:&lt;coord&gt;`, `y:&lt;coord&gt;`, and grouped constraints such as `x:@left`, `x:@+right`, or `@row`.

```
#let src = ```dot
digraph { a -> b [id=0, pin="x:@+right"]; }
```
#edges(layout(parse(src.text).first(), layout-algo: "tree")).first().statements.at("pin",
default: none)
```

Parameters

```
parse(
  input: string bytes,
  default-node-data: any,
  eval-node-fields: string array,
  default-edge-data: any,
  eval-edge-fields: string array,
  default-source-data: any,
  eval-source-fields: string array,
  default-sink-data: any,
  eval-sink-fields: string array,
  eval-graph-fields: string array,
  eval-mode: string,
  scope: dictionary
) -> array
```

input `string` or `bytes`

DOT source text containing one or more `graph` or `digraph` definitions.

default-node-data `any`

Default data merged into every node data. Captured node data fields override it, but bare dot statements don't set data fields. To turn statements into data fields, use `eval-node-fields`.

````
#let a = ```dot
digraph {
a -> b -> c -> d -> a
a -> a
b -> d
c [particle="q"]
}
```
#let g = parse(a.text,default-node-data:(particle:"g")).at(0)
#nodes(g).map(n=>n.data.particle)
````

Default: `none`

eval-node-fields `string` or `array`

Node fields to evaluate into node data. The eval scope includes node statements plus `node`, `name`, and `pos`.

````
#let src = ```dot
digraph { a [label="#str(name)"]; }
```
#nodes(parse(src.text, eval-node-fields: "label").first()).first().data.label
````

Default: `()`

default-edge-data `any`

Default data merged into every edge data. Captured edge data fields override it, but bare dot statements don't set data fields. To turn statements into data fields, use `eval-edge-fields`.

Default: `none`

eval-edge-fields `string` or `array`

Edge statement fields to evaluate into `graph.edges(g).at(i).data`. The eval scope includes edge statements plus `edge`, `orientation`, endpoint records, placement fields, and existing edge data. Drawing callbacks later expose the edge index as `eid`.

````
#let src = ```dot
digraph { a -> b [id=0, label="#orientation"]; }
```
#edges(parse(src.text, eval-edge-fields: "label").first()).first().data.label
````

Default: `()`

default-source-data `any`

Default data merged into every source half-edge data. Captured source data fields override it.

Default: `none`
````

eval-source-fields `string` or `array`

Source half-edge fields to evaluate into `edge.source.data`. The eval scope includes source endpoint fields `statement`, `port-label`, `compass`, `node`, and `hedge`, plus the surrounding edge fields.

```
#let src = ```dot
digraph { a:left:e -> b [id=0, source="#port-label"]; }
```
#edges(parse(src.text, eval-source-fields:
"statement").first()).first().source.data.statement
```

Default: `()`

**default-sink-data** `any`

Default data merged into every sink half-edge data. Captured sink data fields override it.

Default: `none`

**eval-sink-fields** `string` or `array`

Sink half-edge fields to evaluate into `edge.sink.data`. The eval scope includes sink endpoint fields `statement`, `port-label`, `compass`, `node`, and `hedge`, plus the surrounding edge fields.

```
#let src = ```dot
digraph { a -> b:0:w [id=0, sink="#str(hedge)"]; }
```
#edges(parse(src.text, eval-sink-fields:
"statement").first()).first().sink.data.statement
```

Default: `()`

eval-graph-fields `string` or `array`

Graph statement fields to evaluate into `graph.info(g).data`. The eval scope includes graph statements and direct graph record fields.

```
#let src = ```dot
digraph {
  title = "Strong graph";
}
```
#info(parse(src.text, eval-graph-fields: "title").first()).data.title
```

Default: `()`

**eval-mode** `string`

Typst `eval` mode used for string field values.

Default: `"markup"`

**scope** `dictionary`

Additional Typst names available while evaluating field values.

Default: `(:)`
```

node

Create a graph node item for `build`.

A Typst label is the node name used by `source`, `sink`, and `pos`. The optional numeric `id` fixes the resulting graph node index. Extra named arguments are captured as node data fields, so `node(<a>, label: [A], color: red)` stores `(label: [A], color: red)`. The default draw style uses `data.label` as the visible node label when present.

Parameters

```
node(
  ..args: label,
  name: none label,
  id: none int,
  pos: none dictionary,
  shift: none string array dictionary,
  statements: dictionary
) -> array
```

..args `label`

Optional positional node name. Must be a Typst label when provided; extra named arguments become data fields.

name `none` or `label`

Typst node name for references.

Default: `none`

id `none` or `int`

Numeric graph node index. Must be unique and in bounds when provided.

Default: `none`

pos `none` or `dictionary`

Node placement.

Default: `none`

shift `none` or `string` or `array` or `dictionary`

Drawing shift stored as a statement.

Default: `none`

statements `dictionary`

Additional flat node statements. Used by DOT; values cannot nest.

Default: `(:)`

source

Create a source half-edge endpoint.

`node` may be a node name like `<a>` or a numeric node index. `name` gives the half-edge a Typst name; `id` is a numeric half-edge order/index override. Extra named arguments are captured as source data fields.

Parameters

```
source(
  node: label int,
  ..args: any,
  name: none label,
  id: none int,
  statement: none string,
  compass: none string,
  route-points: array
) -> dictionary
```

node `label` or `int`

Referenced node, either by Typst label name or numeric node index.

..args `any`

Extra named arguments become source data fields.

name `none` or `label`

Optional half-edge name used for later references.

Default: `none`

id `none` or `int`

Optional numeric half-edge index/order override.

Default: `none`

statement `none` or `string`

DOT-ish flat statement used for matching/joining dangling half edges.

Default: `none`

compass `none` or `string`

DOT compass point such as `"n"`, `"s"`, `"e"`, or `"w"`.

Default: `none`

route-points `array`

Interior route points ordered from this node toward the edge position. Points accept two-item arrays or `(x:, y:)` dictionaries. They are drawing geometry, not additional graph vertices.

Default: `()`

sink

Create a sink half-edge endpoint.

`node` may be a node name like `<a>` or a numeric node index. `name` gives the half-edge a Typst name; `id` is a numeric half-edge order/index override. Extra named arguments are captured as sink data fields.

Parameters

```
sink(
  node: label int,
  ..args: any,
  name: none label,
  id: none int,
  statement: none string,
  compass: none string,
  route-points: array
) -> dictionary
```

node `label` or `int`

Referenced node, either by Typst label name or numeric node index.

..args `any`

Extra named arguments become sink data fields.

name `none` or `label`

Optional half-edge name used for later references.

Default: `none`

id `none` or `int`

Optional numeric half-edge index/order override.

Default: `none`

statement `none` or `string`

DOT-ish flat statement used for matching/joining dangling half edges.

Default: `none`

compass `none` or `string`

DOT compass point such as `"n"`, `"s"`, `"e"`, or `"w"`.

Default: `none`

route-points `array`

Interior route points ordered from this node toward the edge position. Points accept two-item arrays or `(x:, y:)` dictionaries. They are drawing geometry, not additional graph vertices.

Default: `()`

edge

Create a graph edge item for `build`.

Positional arguments may contain one `source`, one `sink`, and optionally one Typst label used as the edge name. The numeric `id` chooses the edge order. Extra named arguments are captured as edge data fields, so `edge(source(<a>), sink(<b>), weight: 3)` stores `(weight: 3)`. `style` is first-class drawing metadata: `auto` asks `draw` for its configured/default style, a dictionary or callback patches that style, and `none` hides the painted edge. The default draw style uses `data.label` as the visible edge label when present.

Parameters

```
edge(
  ..args: any,
  name: none label,
  id: none int,
  orientation: string,
  pos: none dictionary,
  shift: none string array dictionary,
  label-pos: none string array dictionary,
  label-angle: none int float string,
  bend: none int float string,
  spring-length: none int float,
  style: auto dictionary array function none,
  statements: dictionary
) -> array
```

..args `any`

Source/sink half-edges and optional edge name; extra named arguments become data fields.

name `none` or `label`

Typst edge name.

Default: `none`

id `none` or `int`

Numeric edge order/index override.

Default: `none`

orientation `string`

Edge orientation: `"default"`, `"reversed"`, or `"undirected"`.

Default: `"default"`

pos `none` or `dictionary`

Edge placement.

Default: `none`

shift `none` or `string` or `array` or `dictionary`

Drawing shift stored as a statement.

Default: `none`

label-pos `none` or `string` or `array` or `dictionary`

Edge label position stored as a statement.

Default: `none`

label-angle `none` or `int` or `float` or `string`

Edge label angle stored as a statement.

Default: `none`

bend `none` or `int` or `float` or `string`

Edge bend stored as a statement.

Default: `none`

spring-length `none` or `int` or `float`

Positive finite dimensionless multiplier of each incidence's preferred spring length in force/anneal layout, including dangling-edge doubling. `none` preserves statements/defaults; a missing statement means `1`. Does not set rendered length or change global `spring.length`.

Default: `none`

style `auto` or `dictionary` or `array` or `function` or `none`

Static logical-edge style, callback, `auto` fallback, or `none` to hide.

Default: `auto`

statements `dictionary`

Additional flat edge statements. Used by DOT; values cannot nest.

Default: `(:)`

group

Create a grouped placement coordinate.

`side: "+"` keeps the solved coordinate non-negative and `side: "-"` keeps it non-positive. `start` supplies an initial value without fixing the coordinate; multiple starts for one group are averaged. Groups are layout constraints and therefore require pin placement, which is the `pos` default. Node and edge coordinates with the same group name and side share one axis coordinate throughout layout.

```
#group("right", side: "+", start: 4)
```

Parameters

```
group(
  name: string int bool,
  side: none string,
  start: none int float
) -> dictionary
```

name `string` or `int` or `bool`

Group identifier shared by positions constrained to the same coordinate.

side `none` or `string`

Optional sign constraint: `"+"`, `"-"`, `"positive"`, or `"negative"`.

Default: `none`

start `none` or `int` or `float`

Optional initial coordinate; unlike `pin`, this remains movable.

Default: `none`

pin

Mark one coordinate as a layout constraint.

Parameters

```
pin(value: int float dictionary) -> dictionary
```

value `int` or `float` or `dictionary`

Coordinate value to constrain.

start

Mark one coordinate as an initial layout value only.

Parameters

```
start(value: int float) -> dictionary
```

value `int` or `float`

Coordinate value to use as the layout seed.

pos

Create a first-class graph placement.

The default `mode: "pin"` turns numeric coordinates into layout constraints and also makes the XY coordinates immediately drawable without a layout pass. Use `start(value)` for an individual coordinate

that should only seed the layout, or `pin(value)` to pin an individual numeric coordinate when `mode: "start"` is used. Grouped coordinates are always layout constraints for their axis.

`z` is auxiliary depth for force-based layout only, in the same numeric layout units as x and y (not Typst lengths). Use `pos(z: pin(2))` for a hard raw-depth pin or `pos(z: start(2))` for a movable initial depth. A bare `pos(z: 2)` follows `mode`, defaulting to `"pin"`. The global depth scale and flattening schedule may reduce effective depth to zero without changing a hard raw-depth pin. This is not a rendered third coordinate: node/edge output `pos` and drawing remain 2D. Statements retain the configured raw pin/seed, not the solved free depth, as `"pos-z"` (finite numeric text) and `"pos-z-mode"` (`"pin"` or `"start"`). Other layout backends do not use z.

Both `build` and `map` accept z placements on nodes and edge control points. A z-only placement leaves XY placement and constraints alone, including automatic edge midpoint seeds. Omitting z preserves existing depth metadata. Z groups, relative depth references, `dz`, and non-finite values are not supported; `ref`, `dx`, and `dy` continue to affect XY only.

```
#pos(x: group("right", side: "+"), y: start(10), z: pin(2))
```

Parameters

```
pos(
  x: none int float dictionary,
  y: none int float dictionary,
  z: none int float dictionary,
  ref: none label int,
  dx: none int float,
  dy: none int float,
  mode: string
) -> dictionary
```

x `none` or `int` or `float` or `dictionary`

Absolute or grouped x coordinate.

Default: `none`

y `none` or `int` or `float` or `dictionary`

Absolute or grouped y coordinate.

Default: `none`

z `none` or `int` or `float` or `dictionary`

Auxiliary force-layout depth, numeric or wrapped in `pin` / `start`.

Default: `none`

ref `none` or `label` or `int`

Node reference for relative XY placement, by name or numeric index.

Default: `none`

dx `none` or `int` or `float`

Relative x offset from `ref`.

Default: `none`

dy `none` or `int` or `float`

Relative y offset from `ref`.

Default: `none`

mode `string`

Placement mode: `"pin"` constrains layout, `"start"` only seeds it.

Default: `"pin"`

map

Map graph records with native-data and structural patches. Position patches merge supplied axes: omitted coordinates retain their existing values, pin/start modes and coordinate groups.

`node` and `edge` accept either a callback for every record or a name-keyed dictionary. Dictionary keys are exact string names: `a` matches `<a>`, and `"D1.0"` matches the `<D1.0>` fragment produced by `cut`. Each entry is a constant patch dictionary, a callback, or `none`. Unlisted records and unnamed records are skipped by name-keyed dictionaries. Every supplied name must exist, even when its entry is `none`. Unknown names and invalid mapper, entry, or callback return types are errors. There are no numeric-ID keys, array selectors, wildcard/default selectors, or recursive merges. `graph`, `source`, and `sink` still accept only a callback or `none`.

The callbacks receive the same full decoded records in either form, plus a `fields` dictionary merging native dictionary data, statements, and direct record fields, with later sources taking precedence. A callback returns `none` to leave the record unchanged, `(data: value)` to set new native data, or structural fields such as `pos`, `shift`, `spring-length`, and `statements` to patch data seen by later layout calls. `spring-length` is validated as a positive finite multiplier and stored in the edge statements used by the layout engine. Structural patches are applied separately from native Typst data, so content and stored functions in the existing `data` value are preserved. Unchanged fields in a returned full record are not reapplied; this preserves partial XY placements when patching z or data. Source and sink callbacks may likewise patch `statement`, `port-label`, and `compass`, and `route-points` before subgraph and layout operations run.

Constant patches and callback results use identical semantics. Node structural keys are `pos`, `shift`, and `statements`; edges also accept `label-pos`, `label-angle`, `bend`, and `spring-length`. Half-edge structural fields include `route-points`; an empty array clears its interior route. With no `data` key, other keys shallow-merge into existing native dictionary data (or replace non-dictionary data with that patch). An explicit `data` key instead replaces native data, ignoring other non-structural keys; `data: none` leaves native data unchanged. Native Typst content and stored functions are preserved, not serialized or invoked. A mapper or entry of `none`, an empty name map, an empty patch, or a callback returning `none` leaves the corresponding records unchanged.

```
#let g = build({ node(<a>) })
#let g = map(g, node: node => (data: (label: [A])))
#nodes(g).first().data.label
```

```
#let g = build({
  node(<a>); node(<b>)
  edge(<D1>, source(<a>), sink(<b>), weight: 2)
})
#let g = map(g,
  node: (a: (label: [A]), b: none),
  edge: (D1: e => (weight: e.fields.weight + 1)),
)
#edge-data(g, <D1>).weight
```

Parameters

```
map(
  graph_: dictionary,
  graph: none function,
  node: none function dictionary,
  edge: none function dictionary,
  source: none function,
  sink: none function
) -> dictionary
```

graph_ `dictionary`

Graph object to transform.

graph `none` or `function`

Callback for graph metadata records.

Default: `none`

node `none` or `function` or `dictionary`

Callback for all node records, or name-keyed patches/callbacks/no-ops.

Default: `none`

edge `none` or `function` or `dictionary`

Callback for all edge records, or name-keyed patches/callbacks/no-ops.

Default: `none`

source `none` or `function`

Callback for source half-edge records.

Default: `none`

sink `none` or `function`

Callback for sink half-edge records.

Default: `none`

style

Attach layout-relevant drawing style to a graph.

Node style is measured immediately and stored as `layout-width` / `layout-height` statements for later layout calls. Edge labels are measured as `label-width` / `label-height` statements for label placement. `draw` uses the stored node, edge, endpoint, and label styles by default.

Parameters

```
style(
  graph_: dictionary,
  scope: dictionary,
  unit: int float length ratio,
  node-label: auto content string function none,
  node-label-style: dictionary function,
  node-style: dictionary function none,
  edge-style: dictionary array function none,
  source-style: dictionary array function none,
  sink-style: dictionary array function none,
  edge-label: content string function none,
  edge-label-style: dictionary function
) -> dictionary
```

graph_ `dictionary`

Graph object to style.

scope `dictionary`

Extra scope visible to label/style callbacks.

Default: `(:)`

unit `int` or `float` or `length` or `ratio`

Coordinate length for one graph-layout unit. Numbers are interpreted as em.

Default: `1`

node-label `auto` or `content` or `string` or `function` or `none`

Node label content or callback. `auto` uses node data/statement label or node name.

Default: `auto`

node-label-style `dictionary` or `function`

CeTZ content style for node labels. Its padding contributes to measured node size.

Default: `(:)`

node-style `dictionary` or `function` or `none`

CeTZ node shape style or callback. Explicit numeric radii contribute to measured node size.

Default: `(:)`

edge-style `dictionary` or `array` or `function` or `none`

Logical-edge style dictionary, layers, or callback used as drawing defaults. Explicit draw and record styles override these defaults. `none` hides edges.

Default: `(:)`

source-style `dictionary` or `array` or `function` or `none`

Source endpoint defaults, composed before explicit draw and record styles.

Default: `(:)`

sink-style `dictionary` or `array` or `function` or `none`

Sink endpoint defaults, composed before explicit draw and record styles.

Default: `(:)`

edge-label `content` or `string` or `function` or `none`

Edge label content or callback. `none` leaves edge labels unstyled and unmeasured.

Default: `none`

edge-label-style `dictionary` or `function`

CeTZ content style for edge labels. Its padding contributes to measured edge-label size.

Default: `(:)`

eval-fields

Evaluate selected fields into native data entries.

Each selected field is read from the record's merged `fields` dictionary, evaluated in a scope containing those fields, and written to `data.<field>`.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>), statements: (mom: "p"))
}, default-edge-statements: (display-label: "$#mom$"))
#let g = eval-fields(g, eval-edge-fields: ("display-label",))
#edges(g).first().data.at("display-label")
```

Parameters

```
eval-fields(
  graph_: dictionary,
  eval-graph-fields: string array,
  eval-node-fields: string array,
  eval-edge-fields: string array,
  eval-source-fields: string array,
  eval-sink-fields: string array,
  eval-mode: string,
  scope: dictionary
) -> dictionary
```

graph_ `dictionary`

Graph object whose selected statement fields should be evaluated.

eval-graph-fields `string` or `array`

Graph statement fields to evaluate into `graph.info(g).data`.

Default: `()`

eval-node-fields `string` or `array`

Node statement fields to evaluate into node data.

Default: `()`

eval-edge-fields `string` or `array`

Edge statement fields to evaluate into edge data.

Default: `()`

eval-source-fields `string` or `array`

Source half-edge fields to evaluate into source data.

Default: `()`

eval-sink-fields `string` or `array`

Sink half-edge fields to evaluate into sink data.

Default: `()`

eval-mode `string`

Typst `eval` mode used for string field values.

Default: `"markup"`

scope `dictionary`

Additional Typst names available while evaluating field values.

Default: `(:)`

info

Return graph metadata.

The result has `name`, `global-statements`, `default-edge-statements`, and `default-node-statements`.

```
#let g = build({ node(<a>) }, name: "demo")
#info(g).name
```

Parameters

`info(graph: dictionary) -> dictionary`

graph `dictionary`

Graph object returned by `build`, `parse`, `layout`, or another graph API.

dot

Serialize a graph object to DOT.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>))
}, name: "demo")
#dot(g).contains("digraph demo")
```

Parameters

```
dot(graph: dictionary) -> string
```

graph `dictionary`

Graph object to serialize.

nodes

Return node records, optionally filtered by a subgraph object.

Node `name` values are Typst labels when present.

```
#let g = build({
  node(<a>)
  node(<b>)
})
#nodes(g).map(node => str(node.name)).join(", ")
```

Parameters

```
nodes(
  graph: dictionary,
  subgraph: none dictionary
) -> array
```

graph `dictionary`

Graph object to inspect.

subgraph `none` or `dictionary`

Optional subgraph filter; only nodes incident to selected half edges are returned.

Default: `none`

edges

Return edge records, optionally filtered by a subgraph object.

Edge `name` values are Typst labels when present.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>, compass: "e"), sink(<b>))
})
#edges(g).len()
```

Parameters

```
edges(
  graph: dictionary,
  subgraph: none dictionary
) -> array
```

graph `dictionary`

Graph object to inspect.

subgraph `none` or `dictionary`

Optional subgraph filter; selected edges retain their full endpoint records.

Default: `none`

node-data

Return one named node's native data.

`name` is a Typst label such as `<a>` or the corresponding string name.

```
#let g = build({ node(<a>, label: [A]) })
#node-data(g, <a>).label
```

Parameters

```
node-data(
  graph_: dictionary,
  name: label string
) -> any
```

graph_ `dictionary`

Graph object to inspect.

name `label` or `string`

Node name as a Typst label or its string form.

edge-data

Return one named edge's native data.

`name` is a Typst label such as `<e>` or the corresponding string name.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), <e>, sink(<b>), label: [$p$])
})
#edge-data(g, <e>).label
```

Parameters

```
  edge-data(
    graph_: dictionary,
    name: label string
  ) -> any
```

graph_ `dictionary`

Graph object to inspect.

name `label` or `string`

Edge name as a Typst label or its string form.

update-node-data

Update one named node's native data.

`update` may be a replacement data value or a function `(data, node) => new-data`.

```
#let g = build({ node(<a>) })
#let g = update-node-data(g, <a>, (label: [A]))
#nodes(g).first().data.label
```

Parameters

```
  update-node-data(
    graph_: dictionary,
    name: label string,
    update: any function
  ) -> dictionary
```

graph_ `dictionary`

Graph object to update.

name `label` or `string`

Node name as a Typst label or its string form.

update `any` or `function`

Replacement data or `(data, node) => new-data` callback.

update-edge-data

Update one named edge's native data.

`update` may be a replacement data value or a function `(data, edge) => new-data`.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), <e>, sink(<b>))
})
#let g = update-edge-data(g, <e>, (label: [$p$]))
#edges(g).first().data.label
```

Parameters

```
update-edge-data(
  graph_: dictionary,
  name: label string,
  update: any function
) -> dictionary
```

graph_ `dictionary`

Graph object to update.

name `label` or `string`

Edge name as a Typst label or its string form.

update `any` or `function`

Replacement data or `(data, edge) => new-data` callback.

cut

Open paired edges along a weighted directed half-edge cut.

`left` and `right` are disjoint subgraphs containing opposite halves of each cut edge. Their names identify boundary sides, not underlying source/sink flow or superficial drawing orientation. A half-edge annotation `(winding: n)` requests `n` same-direction seam passages; absent winding defaults to one. If both sides specify a winding they must agree. Net counts of cancelling recrossings are not a supported description of an opening.

The input is unchanged. Every cut edge yields source and sink stubs plus `n - 1` middle segments on auxiliary, unpainted boundary nodes. Fragment names are `<original.0>` through `<original.n>` in underlying source-to-sink order. Unnamed edges use `__linnest_cut_edge_ID` as the prefix; name collisions are errors. Cut geometry is reset for re-layout, while physical data and orientation are retained. Arbitrary Typst data, including content and callbacks, is remapped without serialization. Generated middle endpoint hedges inherit the data of the opposite original hedge, corresponding to the joined boundary side.

Output node/edge/half-edge records have `origin` relative to the input graph. Edge origins contain `(edge, name, segment, winding)`; uncut segments are `none`. Generated nodes have no original node. Boundary nodes and dangling cut edges also have `boundary`: `(edge, node, hedge, side, crossing, data, cut-data, origin)`, where `data` is the side's hedge annotation and `cut-data` its subgraph-wide data. `crossing` counts from zero along underlying flow. Boundary `hedge` and `origin` describe the input to the cut that created that boundary, even after subsequent cuts; `edge` and `node` always locate its current anchor.

This differs from each record's immediate-input `origin`. The optional callback patches only endpoints created by this operation, exactly like `map`; earlier boundary placements and annotations are retained.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), <e>, sink(<b>), label: [$k$])
})
#let left = subgraph.select(g, sink: (<e>,))
#let left = subgraph.with-data(g, left, hedge: (winding: 2))
#let right = subgraph.select(g, source: (<e>,))
#let opened = cut(g, left: left, right: right)
#edges(opened).len()
```

Parameters

```
cut(
  graph_: dictionary,
  left: dictionary,
  right: dictionary,
  boundary: none dictionary function
) -> dictionary
```

graph_ `dictionary`

Graph to open, without modifying it.

left `dictionary`

Selected left half-edges, optionally annotated with winding.

Default: `none`

right `dictionary`

Involution partners defining the right side.

Default: `none`

boundary `none` or `dictionary` or `function`

Placement/data patch for each newly created boundary node or dangling edge.

Default: `none`

boundaries

Inspect all generated cut endpoints with their current solved positions.

Returns the boundary records described by `cut` plus `pos`. For a dangling endpoint, `node` is `none` and `pos` is its edge position; for a middle segment endpoint it is the auxiliary node position. This makes boundary annotations independent of their internal node/edge representation.

Parameters

```
boundaries(graph_: dictionary) -> array
```

graph_ `dictionary`

Graph returned by `cut`, optionally styled, patched, or laid out.

join

Join two graphs by matching dangling half-edge metadata on `key`.

Supported key values are `"statement"`, `"compass"`, `"port-label"`, and `"id"`. The join is partial: unmatched dangling half-edges remain. Matching identities must be unique within each flow, and node/edge names must not collide across the inputs. Native Typst data is preserved; when both sides provide a dictionary, right-only fields are retained and left-side fields take precedence.

```
#let left = build({
  node(<a>)
  edge(sink(<a>, statement: "j"))
})
#let right = build({
  node(<b>)
  edge(source(<b>, statement: "j"))
})
#edges(join(left, right, key: "statement")).len()
```

Parameters

```
join(
  left: dictionary,
  right: dictionary,
  key: string
) -> dictionary
```

left `dictionary`

Left graph object.

right `dictionary`

Right graph object.

key `string`

Dangling half-edge match key: `"statement"`, `"compass"`, `"port-label"`, or `"id"`.

Default: `"statement"`

join-edges

Join two graphs by matching dangling edge records on an edge statement key.

This is distinct from graph.join: it matches edge-level statements rather than half-edge metadata. The same partial-join, collision, and data-merge rules apply.

Parameters

```
join-edges(
  left: dictionary,
  right: dictionary,
  key: string
) -> dictionary
```

left `dictionary`

Left graph object.

right `dictionary`

Right graph object.

key `string`

Nonempty edge statement key used for matching.

Default: `"statement"`

cycles

Return subgraph objects for the graph's cycle basis.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>))
})
#cycles(g).len()
```

Parameters

```
cycles(graph: dictionary) -> array
```

graph `dictionary`

Graph object to analyze.

forests

Return subgraph objects for the graph's spanning forests.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>))
})
#forests(g).len()
```

Parameters

```
forests(graph: dictionary) -> array
```

graph `dictionary`

Graph object to analyze.

Reference

- options()
- layout()
- layout-sequence()

options

Construct reusable semantic layout options or sparsely update existing ones.

Each option group is a plain dictionary. Supplying `base` preserves every field not mentioned by a patch, including fields inside a patched group. Pass the result to `layout` with argument spreading.

```
#let common = options(
  spring: (strength: 8, length: 0.5),
  repulsion: (edge-node: 0.05, dangling: 2),
  solver: (algorithm: "force", steps: 50),
)
#let spacious = options(
  base: common,
  spring: (length: 0.7),
  repulsion: (dangling-centroid: 3),
)
#let g = layout(g, ..spacious)
```

Parameters

```
options(
  ..patches: arguments,
  base: dictionary
) -> dictionary
```

..patches `arguments`

Semantic option-group patches.

base `dictionary`

Existing option dictionary to update.

Default: `(:)`

layout

Apply the linnest layout pass to a graph object.

This is intentionally a second step: construct or parse a graph first, then call `layout`. Set `layout-algo` to `"force"` for deterministic force integration, `"anneal"` for simulated annealing, `"tree"` for a traversal tree placement, `"dot"` for a Graphviz-like layered placement, or `"stable-layered"` for a stable railroad-inspired layered placement.

Force layout uses one `steps` × `epochs` iteration budget and one cooling schedule. Effective auxiliary depth is `scale * raw-z`: `scale` follows a smoothstep from `depth-scale` to zero over the initial `flattening-end` fraction of that budget. Remaining iterations are planar, without restarting `step` or cooling. `early-tol` can stop the run only after scale reaches zero. Unpinned raw depths remain bounded; hard raw-depth pins are

preserved even when their effective depth is zero. Output positions remain 2D, and auxiliary depth does not change draw order; label layout runs after the single graph pass.

```
#let g = graph.parse("digraph partial { a -> b;a -> b;a:s -> b:s; b:s -> c:s; c:s -> d:s;
d:s -> a:s }").at(0)
#let south = subgraph.compass(g,"s")
#let gf = layout(layout(g, layout-algo: "force"), layout-algo: "tree", layout-nodes:
"fixed",subgraph:south)
#let ga = layout(g, layout-algo: "force")
#let gt = layout(layout(g, layout-algo: "tree",layout-roots: (2)), layout-algo: "anneal",
layout-nodes: "fixed",gamma-ee:0.1,gamma-ev:.75,beta:5,length-scale:0.4)
#grid(columns: 3, gutter: 2cm, draw(gf), draw(ga), draw(gt))
```

```
#let g = graph.parse("digraph partial { a -> b; b -> c; c -> d; d -> a }").at(0)
#let tree = graph.forests(g).at(0)
#let g = layout(g, layout-algo: "tree", subgraph: tree)
#graph.edges(g).map(edge => edge.pos)
```

Parameters

```
layout(
  graph: dictionary,
  spring: none dictionary,
  repulsion: none dictionary,
  constraints: none dictionary,
  labels: none dictionary,
  solver: none dictionary,
  subgraph: none dictionary,
  viewport-w: float,
  viewport-h: float,
  tree-dx: float,
  tree-dy: float,
  steps: int,
  seed: int,
  step: float,
  step-shrink: float,
  cool: float,
  accept-floor: float,
  early-tol: float,
  temp: float,
  delta: float,
  beta: float,
  k-spring: float,
  spring-length-scale: float,
  initial-repulsion: float,
  repulsion-growth: float,
  g-center: float,
  epochs: int,
  crossing-penalty: float,
  gamma-dangling: float,
  gamma-dangling-centroid: float,
  external-pull: float,
  external-pull-balance: float,
  external-pull-attachment: float,
  gamma-ee: float,
  directional-force: float,
  internal-label-length-scale: float,
  external-label-length-scale: float,
  label-spring: float,
  label-charge: float,
  label-steps: int,
  label-layout: string,
  label-step: float,
  label-early-tol: float,
  label-max-delta-scale: float,
  gamma-ev: float,
  eps: float,
  incremental-energy: bool,
  layout-algo: string,
  layout-nodes: string,
  layout-direction: string,
  rank-align: string,
  layout-roots: array,
  rank-same: array,
  route-edge-weight: float,
  route-exit-weight: float,
  route-label-width-scale: float,
  route-label-width-cap: float,
  depth-scale: float,
  flattening-end: float,
  length-scale: float,
  impred-steps: int,
  impred-step-scale: int,
  impred-level: bool,
```

graph `dictionary`

Graph object returned by `graph.build` or `graph.parse`.

spring `none` or `dictionary`

Semantic incidence-spring options. Supported fields are `strength` and `length`. These override the corresponding flat parameters below.

Default: `none`

repulsion `none` or `dictionary`

Semantic repulsion options. Supported fields are `strength`, `centering`, `edge-node`, `edge-edge`, `dangling`, `dangling-centroid`, and `softening`. These override the corresponding flat parameters below.

Default: `none`

constraints `none` or `dictionary`

Semantic constraint options. Supported fields are `side-strength`, `node-movement`, `direction`, `rank-alignment`, `roots`, and `same-rank`. These override the corresponding flat parameters below.

Default: `none`

labels `none` or `dictionary`

Semantic label-layout options. Supported fields are `internal-distance`, `external-distance`, `spring`, `repulsion`, `steps`, `model`, `step`, `tolerance`, and `max-movement`. These override the corresponding flat parameters below.

Default: `none`

solver `none` or `dictionary`

Semantic solver options. Supported fields are `algorithm`, `steps`, `epochs`, `seed`, `step`, `step-shrink`, `cooling`, `acceptance-floor`, `tolerance`, `temperature`, `max-movement`, `incremental-energy`, `subgraph-mode`, `crossing-penalty`, `depth-scale`, and `flattening-end`. These override the corresponding flat parameters below.

Default: `none`

subgraph `none` or `dictionary`

Optional subgraph object to lay out. With `"tree"`, other edges are drawn from the resulting node positions. With `"dot"` and `"stable-layered"`, the subgraph determines rank constraints, while all paired edges between included nodes get dummy routing vertices and edge positions. With `"force"` and `"anneal"`, `solver.subgraph-mode: "fixed-boundary"` keeps outside points fixed but interacting; `"isolated"` excludes them from the solver and label-layout interactions. The selection must have compatible topology, and isolated selections cannot cross grouped coordinates.

Default: `none`

viewport-w `float`

Width of the layout viewport used to derive the natural spring length. Applies to both `"force"` and `"anneal"`.

Default: `10.0`

viewport-h `float`

Height of the layout viewport used to derive the natural spring length. Applies to both `"force"` and `"anneal"`.

Default: `10.0`

tree-dx `float`

Horizontal spacing multiplier for traversal-tree and layered placement. For `"force"` and `"anneal"`, this scales the initial placement.

Default: `0.9`

tree-dy `float`

Vertical spacing multiplier for traversal-tree and layered placement. For `"force"` and `"anneal"`, this scales the initial placement.

Default: `1.2`

steps `int`

Iterations per epoch. In `"force"` mode this is the number of force integration steps within the single depth-flattening schedule; in `"anneal"` mode this is the number of proposals per temperature epoch.

Default: `100`

seed `int`

Seed for deterministic initialization, force-mode jitter, and annealing proposals. Applies to both modes.

Default: `2`

step `float`

Initial movement scale. `"force"` multiplies computed forces by this value, with no restart when depth reaches zero; `"anneal"` uses it as a proposal step size in natural spring-length units.

Default: `0.81`

step-shrink `float`

Anneal-only step shrink factor, applied when an epoch's acceptance ratio falls below `accept-floor`.

Default: `0.21`

cool `float`

Cooling factor applied once per epoch. `"anneal"` cools `temp`; `"force"` shrinks `step`.

Default: `0.85`

accept-floor `float`

Anneal-only acceptance-ratio threshold below which `step` is shrunk by `step-shrink`.

Default: `0.15`

early-tol `float`

Force-only early stop threshold for maximum movement in one step, as a multiple of the natural spring length, eligible only after effective depth scale reaches zero and repulsion reaches its final strength. The annealing schedule stores this value but does not currently use it for stopping.

Default: `1e-6`

temp `float`

Anneal-only initial temperature used in the Metropolis acceptance test, scaled by natural spring length squared.

Default: `0.3`

delta `float`

Force-mode maximum movement clamp per point and per step, as a multiple of the natural spring length.

Default: `0.4`

beta `float`

Base repulsion strength for vertex-vertex interactions. Also scales `gamma-ev`, `gamma-ee`, `gamma-dangling`, `gamma-dangling-centroid`, and `g-center`. Applies to both modes through the shared spring energy.

Default: `50.0`

k-spring `float`

Spring stiffness for node-to-edge incidence lengths. Applies to both modes.

Default: `11.0`

spring-length-scale `float`

Rest-length multiplier, independent of drawing scale and repulsive strengths. Multiplies each edge's `spring-length` value.

Default: `1.0`

initial-repulsion `float`

Initial fraction of final repulsion for force layout. It grows monotonically without resetting positions; 1 disables continuation.

Default: `1.0`

repulsion-growth `float`

Fraction of the iteration budget spent growing repulsion to its final value. Remaining iterations settle at full repulsion.

Default: `0.7`

g-center `float`

Centering strength relative to `beta`. Applies to both modes.

Default: `0.002`

epochs `int`

Number of epochs. Both modes run up to `steps` iterations inside each epoch. Force mode shares this budget and its cooling schedule across depth flattening and the remaining planar relaxation.

Default: `30`

crossing-penalty `float`

Fixed energy penalty per detected edge crossing. Force layout also uses it when choosing swaps along constrained lines during planar relaxation; it does not add a continuous crossing force.

Default: `30.0`

gamma-dangling `float`

Repulsion for dangling half edges, relative to `beta`. Applies to both modes through the shared spring energy.

Default: `5.0`

gamma-dangling-centroid `float`

Repulsion of every dangling endpoint from the current node centroid, relative to `beta`. The equal-and-opposite reaction is shared over the nodes, avoiding translational drift. Applies to both modes.

Default: `0.0`

external-pull `float`

Constant external pull, scaled by `k-spring` and natural spring length. Mixed momentum directions pull left/right; one direction spreads radially. Components with both directions balance their total pull across the sides and normalize it by the largest unit-flow spring load, including dangling springs. A series chain with one leg at each end keeps its pull regardless of length; parallel routes share the load. Distributed owners sharing an external X coordinate receive extra pull to clear their outermost owner, which can exceed the base force. One-direction components keep the base force. These fixed topology weights apply identically to force and energy. Reaction is shared over nodes, preserving translation and respecting pins. Zero disables the pull. Applies to force and annealing modes.

Default: `0.0`

external-pull-balance `float`

Non-negative exponent on topology weights: 0 gives uniform per-endpoint pull, 1 keeps the default balancing, and values above 1 strengthen it. One-direction radial pull is unaffected.

Default: `1.0`

external-pull-attachment `float`

Non-negative multiplier on the extra demand from distributed owners sharing an external X coordinate. Zero uses only bottleneck load sharing; 1 keeps the default correction, and larger values increase that demand. Terminal groups and one-direction radial pull are unaffected.

Default: `1.0`

gamma-ee `float`

Local edge-edge repulsion, relative to `beta`. Applies to both modes.

Default: `0.1`

directional-force `float`

Bias that pushes points in directions implied by pin/port constraints. Force mode treats this as a force and scales it by natural spring length; anneal mode treats it as a dimensionless multiplier on proposal steps. Applies to both modes.

Default: `5.0`

internal-label-length-scale `float`

Paired-edge label target offset as a multiple of the graph spring length. Label layout runs after both graph layout modes.

Default: `0.6`

external-label-length-scale `float`

Dangling-edge label target offset as a multiple of the graph spring length. Independent of internal label spacing.

Default: `0.6`

label-spring `float`

Spring strength pulling each label toward its target offset in the spring-based label layouts.

Default: `23.0`

label-charge `float`

Repulsion strength between labels and graph points, scaled by spring length squared. Label layout runs after both modes.

Default: `3.0`

label-steps `int`

Maximum number of post-layout label relaxation steps. Set to `0` to skip label placement. Applies after both modes.

Default: `20`

label-layout `string`

Edge-label relaxation model. `"normal"` uses a perpendicular offset, `"dangling-tangent"` uses the edge direction for dangling half-edge labels and a perpendicular offset for paired edges, and `"fixed-length"` keeps each label at a fixed distance from its edge point and only lets that segment rotate. `"fixed-gap"` keeps internal labels at a uniform clearance from their measured text box, sliding along the rendered curve to avoid overlaps without changing that gap; dangling labels retain outward tangent relaxation. ImPrEd relaxes no labels: in every mode the drawing's annotation search places its internal labels, clearing coil and wave bands.

Default: `"normal"`

label-step `float`

Label relaxation step size. Applies after both modes.

Default: `0.15`

label-early-tol `float`

Label relaxation early stop threshold. Applies after both modes.

Default: `1e-3`

label-max-delta-scale `float`

Label movement clamp as a multiple of spring length. Applies after both modes.

Default: `0.5`

gamma-ev `float`

Edge-vertex repulsion, relative to `beta`. Applies to both modes.

Default: `0.01`

eps `float`

Softening epsilon used in inverse-square force/energy terms. Applies to both modes.

Default: `1e-4`

incremental-energy `bool`

Whether annealing updates cached energy by local deltas. This is anneal-only; force mode computes direct forces instead of energies.

Default: `true`

layout-algo `string`

Layout algorithm. Use `"force"` for direct force integration, `"anneal"` for simulated annealing against the spring energy, `"tree"` for a traversal-tree placement, `"dot"` for a Graphviz-like layered placement, `"stable-layered"` for a stable railroad-inspired layered placement, or `"impred"` for constrained planarization and topology-preserving relaxation.

Default: `"force"`

layout-nodes `string`

Node movement policy. `"layout"` lets the layout algorithm move nodes. `"fixed"` keeps every node at its current position for this layout pass and only moves edge control points. With `subgraph`, only edges in the subgraph are moved; other edge control points stay at their current positions.

Default: `"layout"`

layout-direction `string`

Direction for traversal-tree and layered rank placement. `"down"` places increasing ranks downward; `"right"` swaps the layout axes so increasing ranks go left-to-right and measured node/label widths reserve rank-axis space.

Default: `"down"`

rank-align `string`

Alignment of real nodes inside a rank along the rank axis. `"center"` keeps node centers aligned, while `"start"` / `"left"` and `"end"` / `"right"` align the corresponding measured node-box side.

Default: `"center"`

layout-roots `array`

Ordered node indices used as preferred roots for `"tree"`, `"dot"`, and `"stable-layered"`. Roots outside the selected node set are ignored. Remaining components are laid out afterward in graph order.

Default: `()`

rank-same `array`

Subgraph objects whose incident nodes should share a dot/stable-layered rank; node-index arrays and callbacks receiving the graph are also accepted. These are layout hints supplied by Typst rather than parsed graph structure.

Default: `()`

route-edge-weight `float`

Dot/stable-layered also honors a node statement `layout-rank` as an exact non-negative integer rank. Nodes with the same `layout-rank` are placed on the same horizontal layer, and larger ranks are placed lower. Relative layout weight for paired edges outside the dot/stable-layered rank subgraph. Lower values make these edges guide routing without dominating the rank tree.

Default: `0.15`

route-exit-weight `float`

Extra horizontal straightening weight for the first or last segment of an edge with `source-route-exit` or `sink-route-exit` set to a vertical side.

Default: `4.0`

route-label-width-scale `float`

Multiplier for measured edge-label width when sizing non-rank dummy routing vertices in dot/stable-layered layout.

Default: `1.0`

route-label-width-cap `float`

Maximum non-rank dummy label width as a multiple of `tree-dx`. Set to `0` or a negative value to disable the cap.

Default: `2.0`

depth-scale `float`

Force-only initial multiplier of raw auxiliary depth, finite and >= 0. Effective z is this smoothly decreasing scale times raw z, not a spring pulling raw coordinates toward the plane. A value of 0 starts in 2D; flattening preserves hard raw-depth pins while removing their invisible separation from the visible 2D force calculation.

Default: `1.0`

flattening-end `float`

Force-only fraction of the total `steps` × `epochs` iteration budget over which the depth scale reaches zero by smoothstep, finite and in 0..=1. Zero starts in 2D; 1 still evaluates the final iteration on the exact plane. Neither value adds a phase or restarts the cooling schedule.

Default: `0.5`

length-scale `float`

Natural spring-length multiplier. This scales the graph's preferred edge length and dimensional force/energy terms so changing only this value mostly zooms the result instead of retuning the force ratios. Applies to both modes.

Default: `0.35`

impred-steps `int`

ImPrEd nominal cooling budget; accelerated integration can use fewer solves.

Default: `500`

impred-step-scale `int`

Maximum virtual-iteration stride after the first 20% of fine steps. Must be positive; 1 retains the reference integration schedule.

Default: `2`

impred-level `bool`

Rotate the drawing to its external-pull optimum at every refinement checkpoint, levelling mixed incoming/outgoing legs. Pinned or aligned coordinates keep their frame.

Default: `true`

impred-spacing `float`

ImPrEd target route spacing.

Default: `2.4`

impred-repulsion `float`

ImPrEd point repulsion multiplier.

Default: `2.5`

impred-attraction `float`

ImPrEd route attraction multiplier.

Default: `2.5`

impred-parallel-balance `float`

Correction for multiple parallel edges.

Default: `1.0`

impred-pull `float`

Topology-normalized external pull.

Default: `0.45`

impred-pull-balance `float`

External pull normalization exponent.

Default: `1.0`

impred-pull-attachment `float`

Pull transmitted to constrained external attachments.

Default: `4.0`

impred-external-max-points `int`

Maximum intermediate points on each external route, from zero to three.

Default: `2`

impred-split-length-ratio `float`

Segment length threshold for subdivision, relative to spacing.

Default: `1.5`

impred-contract-chord-ratio `float`

Chord length threshold for route contraction, relative to spacing.

Default: `1.25`

impred-edge-clearance `float`

ImPrEd node/edge clearance.

Default: `0.4`

impred-node-edge-strength `float`

ImPrEd point-to-segment repulsion multiplier.

Default: `4.0`

impred-labels `bool`

Opt-in label-aware layout: lay out each measured internal label as a point tethered beside its carrier, so the drawing makes room for it; the annotation search then prefers that side. Off by default.

Default: `false`

layout-sequence

Apply multiple layout passes in order.

Each pass is a dictionary of named arguments accepted by `layout`, excluding the graph itself.

Parameters

```
layout-sequence(
  graph: dictionary,
  passes: array
) -> dictionary
```

graph `dictionary`

Graph object returned by `graph.build` or `graph.parse`.

passes `array`

Array of layout option dictionaries.

Reference

- edge-halves()
- to-cetz-edge-halves()
- draw()

edge-halves

Split a laid-out graph edge into source and sink half-edge paths.

The returned dictionary has `source`, `sink`, `curve`, and `split-gap`. The path follows the source route points, edge layout point, and reversed sink route points. The split point is the edge layout point, so a zero-gap pair joins smoothly there. Repeated consecutive routed points are collapsed for drawing. `split-gap` reports the effective gap when a requested half-gap is longer than either half-edge.

Parameters

```
edge-halves(
  edge: dictionary,
  nodes: array,
  omega: float,
  source-outset: int float,
  sink-outset: int float,
  split-gap: int float,
  accuracy: float
) -> dictionary
```

edge `dictionary`

Edge record returned by `graph.edges(layout(g))`.

nodes `array`

Node records returned by `graph.nodes(layout(g))`.

omega `float`

Hobby curl used for the source-to-edge and edge-to-sink curves.

Default: `1.0`

source-outset `int` or `float`

Arc-length trim applied at the source node side.

Default: `0`

sink-outset `int` or `float`

Arc-length trim applied at the sink node side.

Default: `0`

split-gap `int` or `float`

Total arc-length gap centered on the edge layout point.

Default: `0`

accuracy `float`

Arc-length accuracy used while trimming.

Default: `0.001`

to-cetz-edge-halves

Draw the two halves of a laid-out graph edge through CeTZ.

`source-style` applies from the source node to the edge layout point, and `sink-style` applies from the edge layout point to the sink node.

Parameters

```
to-cetz-edge-halves(
  edge: dictionary,
  nodes: array,
  unit: int float length ratio,
  omega: float,
  source-outset: int float,
  sink-outset: int float,
  split-gap: int float,
  accuracy: float,
  source-style: dictionary,
  sink-style: dictionary
) -> content
```

edge `dictionary`

Edge record returned by `graph.edges(layout(g))`.

nodes `array`

Node records returned by `graph.nodes(layout(g))`.

unit `int` or `float` or `length` or `ratio`

Coordinate length for one graph-layout unit. Numbers are interpreted as em.

Default: `1`

omega `float`

Hobby curl used for the source-to-edge and edge-to-sink curves.

Default: `1.0`

source-outset `int` or `float`

Arc-length trim applied at the source node side.

Default: `0`

sink-outset `int` or `float`

Arc-length trim applied at the sink node side.

Default: `0`

split-gap `int` or `float`

Total arc-length gap centered on the edge layout point.

Default: `0`

accuracy `float`

Arc-length accuracy used while trimming.

Default: `0.001`

source-style `dictionary`

CeTZ style for the source half edge.

Default: `(:)`

sink-style `dictionary`

CeTZ style for the sink half edge.

Default: `(:)`

draw

Draw a graph object with CeTZ.

The graph must already have positions, either from `layout` or from explicit `pos` values passed to graph node or edge items. Paired edges without an explicit edge position use the midpoint of their endpoint nodes. Node and edge Typst style dictionaries or callbacks are evaluated and forwarded to CeTZ.

```
#let positioned = graph.build({
  graph.node(<left>, label: [left], pos: graph.pos(x: 0, y: 0))
  graph.node(<right>, label: [right], pos: graph.pos(ref: <left>, dx: 2.5, dy: 0))
  graph.edge(graph.source(<left>), graph.sink(<right>))
  graph.edge(graph.source(<right>), <right-out>, pos: graph.pos(ref: <right>, dx: 0.9, dy: 0.7))
},
  name: "positioned",
)
#draw(positioned, source-style: (stroke: black + 0.7pt), sink-style: (stroke: black + 0.7pt))
```

```
#let parallel-base-edge = 0
#let source-patterns = (
  "coil",
  "zigzag",
  "coil",
  "wave",
  "zigzag",
  "coil",
  "wave",
  "zigzag",
)
#let sink-patterns = (
  "coil",
  "zigzag",
  "coil",
  "zigzag",
  "coil",
  "wave",
  "coil",
  "wave",
```

```
)
#let parallel-layer(edge, mark: none) = if edge.eid == parallel-base-edge {
  (
    offset: -0.5,
    length: 1.5,
    ratio: 0.5,
    resolve-length: "min",
    stroke: (paint: rgb("#2f6f4e"), thickness: 1.1pt, cap: "round"),
    mark: mark,
  )
} else { none }
#let stack(base, layer) = if layer == none { base } else { (base, layer) }
#let source-stroke(edge) = if edge.eid == parallel-base-edge {
  (paint: gray, thickness: 0.75pt, cap: "round")
} else {
  (paint: red, thickness: 1.2pt, cap: "round")
}
#let sink-stroke(edge) = if edge.eid == parallel-base-edge {
  (paint: gray, thickness: 0.75pt, cap: "round")
} else {
  (paint: blue, thickness: 1.2pt, cap: "round", dash: "dotted")
}
#let source-style(edge) = {
  let base = (
    stroke: source-stroke(edge),
    pattern: source-patterns.at(edge.eid),
    pattern-amplitude: 0.18,
    pattern-wavelength: 0.55,
    pattern-coil-longitudinal-scale: 1.6,
  )
  stack(base, parallel-layer(edge))
}
#let sink-style(edge) = {
  let base = (
    stroke: sink-stroke(edge),
    pattern: sink-patterns.at(edge.eid),
    pattern-amplitude: 0.18,
    pattern-wavelength: 0.55,
    pattern-coil-longitudinal-scale: 1.6,
  )
  stack(base, parallel-layer(edge, mark: curve.mark.straight(length: 0.15cm, width: 0.1125cm)))
}
#let g = graph.build({
  graph.node(<a>, label: [a hi])
  graph.node(<c>)
  graph.node(<d>)
  graph.node(<e>)
  graph.edge(graph.source(<a>), <ac>, graph.sink(<c>), pos: graph.pos(x: 0, y: 0.75, mode:
"pin"))
  graph.edge(graph.source(<c>), <ca>, graph.sink(<a>, compass: "e"))
  graph.edge(graph.source(<c>), <cd>, graph.sink(<d>, compass: "e"))
  graph.edge(graph.source(<e>), <ed>, graph.sink(<d>, compass: "e"))
  graph.edge(graph.source(<e>), <ea>, graph.sink(<a>, compass: "e"))
  graph.edge(graph.source(<d>), <d-out>)
  graph.edge(graph.source(<e>, compass: "e"), <e-out>)
  graph.edge(graph.source(<a>), <a-out>)
},
  name: "demo",
)
#let layed-out = layout(g)
#let east = subgraph.compass(layed-out, "e")
#draw(layed-out, subgraph: east, source-style: source-style, sink-style: sink-style)
```

```
#let p = graph.build({
  graph.node(<pa>, label: [a], pos: graph.pos(y: 0, mode: "pin"))
  graph.node(<pc>, label: [c], pos: graph.pos(y: 0, mode: "pin"))
  graph.node(<pc1>, label: [c], pos: graph.pos(y: 0, mode: "pin"))
  graph.node(<pc2>, label: [c], pos: graph.pos(y: 0, mode: "pin"))
  graph.edge(graph.source(<pa>), <e0>, graph.sink(<pc>))
  graph.edge(graph.source(<pc2>), <e1>, graph.sink(<pc1>))
},
  name: "parallel demo",
)
#let parallel-edge-style(edge) = (
    offset: -0.5,
    length: 1.4,
    ratio: 0.5,
    resolve-length: "min",
    stroke: (paint: rgb("#2f6f4e"), thickness: 1.1pt, cap: "round"),
  )

#let focused-base-style(edge) = (
  stroke: (paint: gray, thickness: 0.7pt, cap: "round"),
  pattern: "coil",
  pattern-amplitude: 0.14,
  pattern-wavelength: 0.45,
  pattern-coil-longitudinal-scale: 1.5,
)
#let focused-source-style(edge) = (focused-base-style(edge), parallel-edge-style(edge))
#let focused-sink-style(edge) = (focused-base-style(edge), parallel-edge-style(edge) + (
  mark: curve.mark.triangle(length: 0.15cm, width: 0.1125cm),
))
#draw(layout(p, g-center: 0.004, label-steps: 0,), unit: 1.25, source-style: focused-source-
style, sink-style: focused-sink-style)



#let g = graph.build({
  graph.node(<a>, pos: graph.pos(x: 0, y: 0, mode: "pin"))
  graph.node(<c>, pos: graph.pos(x: 3, y: 0, mode: "pin"))
  graph.node(<d>, pos: graph.pos(x: 3, y: -1, mode: "pin"))
  graph.edge(graph.source(<a>), <ac>, graph.sink(<c>), orientation: "default")
  graph.edge(graph.source(<a>), <ad>, graph.sink(<d>), orientation: "reversed")
},
  name: "oriented marks",
)
#let arrow = curve.mark.triangle(length: 0.15cm, width: 0.1125cm)
#let oriented-arrow = (
  stroke: black + 0.7pt,
  mark: arrow,
  mark-position: "center-if-dangling",
  mark-orientation: "edge",
)
#draw(layout(g), unit: 1.4, source-style: oriented-arrow, sink-style: oriented-arrow)
```

A style layer may set `crossing-under` to a visible edge's integer id or Typst label name and `crossing-gap` (default `0.55`) to the total arc length hidden around each proper centerline intersection. References are resolved independently of edge order. The current layer is split with Kurvst, so patterned phases continue across the hidden spans. A mark is emitted once: its station uses the concatenated visible arc length, and its fit applies to the connected visible fragment containing that station. Dangling layers and paired layers with one continuous source/sink paint style are supported. On a paired layer both half styles must specify the same target and gap. The cut layer cannot participate in a subgraph underlay. References to self, unknown, or invisible edges are errors. A valid pair with no proper interior intersection is left unchanged.

SVG inspection normally uses the drawn graph's IDs. Display transformations may also supply `title`, `summary`, and `properties` in `inspection` data. `summary` supplies the hover text; `title` names the click inspector and `properties` is an array of `(label, value)` string pairs shown as detail rows. These fields augment the standard topology and selection information. Display transformations can retain a source graph's identities with native `inspection` data: nodes accept `(node, edges)` and edges accept `(edge, source, sink, source-hedge, sink-hedge)`. These override inspection and selection only; layout, drawing callbacks, and subgraphs still use the drawn graph's IDs.

Parameters

```
draw(
  graph: dictionary,
  scope: dictionary,
  unit: auto int float length ratio,
  title: none auto content string,
  subgraph: none dictionary array,
  debug: bool int,
  debug-label-collisions: bool,
  label-collision-padding: int float,
  external-label-gap: int float,
  show-half-edge-ids: bool,
  node-radius: auto int float array,
  node-min-radius: int float,
  node-label-padding: int float,
  node-fill: any,
  node-stroke: any,
  node-outset: auto int float,
  node-label-style: dictionary,
  node-style: dictionary function none,
  node-label: auto content string function none,
  draw-node: auto function,
  draw-after: none array function,
  edge-stroke: any,
  edge-style: dictionary array function none,
  edge-offset: int float,
  edge-length: none int float,
  edge-ratio: none int float,
  edge-resolve-length: string function,
  edge-accuracy: float,
  edge-split-gap: int float,
  edge-dangling-tangent: auto string,
  source-style: dictionary array function none,
  sink-style: dictionary array function none,
  edge-label: content string function none,
  edge-label-style: dictionary function none,
  edge-omega: float,
  edge-trim-accuracy: float,
  padding: none int float array dictionary,
  debug-edge-radius: int float,
  debug-edge-fill: any,
  debug-edge-stroke: any,
  debug-edge-label-fill: any,
  subgraph-edge-style: dictionary,
  subgraph-edge-underlay: bool
) -> content
```

graph `dictionary`

Graph object with positions from `layout` or explicit graph API `pos` fields.

scope `dictionary`

Additional values merged into node and edge callback dictionaries.

Default: `(:)`

unit `auto` or `int` or `float` or `length` or `ratio`

Coordinate length for one graph-layout unit. `auto` uses any unit stored by `graph.style`, falling back to `1em`. Numbers are interpreted as em.

Default: `auto`

title `none` or `auto` or `content` or `string`

Optional title displayed above the diagram. Use `auto` for the graph name.

Default: `none`

subgraph `none` or `dictionary` or `array`

Optional subgraph or array of subgraphs whose half-edges are shaded. Array entries may be subgraph objects or records like `(subgraph: sg, edge-style: (stroke: red + 2pt))`. Each selection must have topology compatible with the graph.

Default: `none`

debug `bool` or `int`

Debug level. `1` enables CeTZ canvas debug; `2` also marks edge positions.

Default: `false`

debug-label-collisions `bool`

Show the padded boxes used by label placement: dashed purple for label–label clearance, cyan for label–obstacle clearance, and orange for node/edge obstacles. Base pair padding is shared equally; label boxes also include `label-collision-padding`. Ordinary painted edges penalize only intersections with the unpadded text box; self-loop bends retain clearance boxes. Green boxes show momentum shafts and arrowheads. Floating overlays do not affect placement, canvas bounds, or interaction targets.

Default: `false`

label-collision-padding `int` or `float`

Extra padding on every side of label collision boxes, in canvas units. Adds to the base label–label and label–obstacle clearances; zero restores those base boxes. Affects sliding/side choices, not the normal label gap or the text–edge intersection test. Arrow and text slide together; coupled moves help crowded annotations pass one another.

Default: `_impl-label-collision-padding.extra`

external-label-gap `int` or `float`

Clearance from a dangling endpoint to the nearest side of its automatic label's text box, in canvas units, measured along the outward tangent. Explicit label positions and path-attached labels keep their placement.

Default: `0.25`

show-half-edge-ids `bool`

Show `h_i` beside every half-edge without an explicit endpoint label.

Default: `false`

node-radius `auto` or `int` or `float` or `array`

Default CeTZ node radius. Use `auto` to fit the node label.

Default: `auto`

node-min-radius `int` or `float`

Minimum radius used when `node-radius` is `auto`.

Default: `0.16`

node-label-padding `int` or `float`

Extra canvas-unit padding around labels when `node-radius` is `auto`.

Default: `0.08`

node-fill `any`

Default CeTZ node fill.

Default: `white`

node-stroke `any`

Default CeTZ node stroke.

Default: `black`

node-outset `auto` or `int` or `float`

Edge clearance from node centers. `auto` uses each node circle radius. Increase this when labels extend beyond the circle.

Default: `auto`

node-label-style `dictionary`

Default node-label style forwarded to `cetz.draw.content`.

Default: `(:)`

node-style `dictionary` or `function` or `none`

Node style dictionary or callback. A callback receives node data.

Default: `(:)`

node-label `auto` or `content` or `string` or `function` or `none`

Node label content or callback. `auto` uses the node name.

Default: `auto`

draw-node `auto` or `function`

CeTZ-compatible node drawing callback. `auto` draws the default circular node and label. A callback receives `(node, box)` and should return CeTZ draw elements; `box` contains `name`, `center`, `width`, `height`, `unit`, `label`, `label-style`, `style`, `radius`, and `node`. Generated cut boundary nodes skip this callback and all node painting; they retain a named position and zero-size anchor box for incident edges.

Default: `auto`

draw-after `none` or `array` or `function`

Additional CeTZ elements drawn after the edges, labels, and nodes in the same canvas. A callback receives `(graph, bounds)` and returns CeTZ elements. `bounds` contains numeric `left`, `right`, `top`, `bottom`, `width`, and `height` in graph units, enclosing the rendered graph before overlays and canvas padding. It includes shapes, curves, and label boxes; CeTZ does not add stroke thickness to path bounds. `top` is the greatest y coordinate, and all fields are zero for an empty drawing. Use `graph.nodes` / `graph.edges` to inspect the positioned graph. These elements share `unit` and contribute to canvas bounds, but do not participate in graph layout.

Default: `none`

edge-stroke `any`

Default CeTZ edge stroke.

Default: `0.1em`

edge-style `dictionary` or `array` or `function` or `none`

Default logical-edge style dictionary, layers, or callback. An edge's `style:` value patches this default; `auto` delegates and `none` hides the edge without changing its topology. Stored `graph.style` edge and endpoint defaults are composed first, followed by this style, record edge style, draw endpoint style, and endpoint-local style. Shared logical-edge callbacks run once before the two halves are composed.

Default: `(:)`

edge-offset `int` or `float`

Default normal offset for edge paths. Applied to the base edge geometry before patterns; node outsets then trim the shifted path.

Default: `0`

edge-length `none` or `int` or `float`

Maximum visible arc length for centered parallel edge paths. `none` keeps the full shifted path.

Default: `none`

edge-ratio `none` or `int` or `float`

Maximum visible fraction of the full offset path length for centered edge paths. Combined with `edge-length` according to `edge-resolve-length`.

Default: `none`

edge-resolve-length `string` or `function`

Resolve `edge-length` and `edge-ratio`. Accepted string values are `"min"`/`"shorter"`, `"max"`/`"longer"`, `"length"`/`"fixed"`, `"ratio"`/`"relative"`, or `"none"`/`"full"`. A function receives one dictionary with `base-length`, `length`, and `ratio`. `base-length` is the offset path length; `length` and `ratio` are positive absolute lengths or `none`. Unknown strings are rejected.

Default: `"min"`

edge-accuracy `float`

Arc-length accuracy for fitted parallel edge paths.

Default: `0.001`

edge-split-gap `int` or `float`

Total arc-length gap centered on the source/sink split at the edge layout point. A per-layer `split-gap` overrides this value.

Default: `0`

edge-dangling-tangent `auto` or `string`

Tangent constraint at dangling edge positions. `"horizontal"` and `"vertical"` preserve `bend` while making the corresponding endpoint tangent exact. A per-layer `dangling-tangent` overrides this value.

Default: `auto`

source-style `dictionary` or `array` or `function` or `none`

Source half-edge style dictionary, array of layer dictionaries, or callback. A `"center"` or numeric mark on a common source/sink paint layer is placed once on the complete derived layer, so it follows `shift`. `mark-position` also accepts a numeric path-length ratio from `0` to `1`; `mark-shift` moves a centered or numeric mark by signed arc length, with positive values moving toward the derived path's end. This also works on patterned dangling and asymmetric paired layers without resetting pattern phase. `mark-position: "center-if-dangling"` centers dangling marks while retaining the paired-edge split point selected by `mark-orientation`. `mark-orientation: "edge"` makes a mark follow `edge.orientation` instead of raw path direction; reversed edges move the mark to the sink half and point it backward, while undirected edges suppress it. `mark` accepts only data-only `curve.mark` specifications, not CeTZ mark dictionaries. Stations use the derived visible carrier after offsets, outsets and crossing windows, before mark shortening. Interior heads straddle their station on a chord. End heads default to `fit: "chord"`; `fit: "bend"` retracts the endpoint along its tangent, retaining controls, by the mark's end distance times `shorten` (default 100%). Short carriers retain full-size heads and clamp the shaft contact independently. Patterned strokes follow the resulting painted shaft. Composite marks fit once as a whole. `source-anchor` may be a CeTZ anchor name such as `"north"` or `"south"` to route this endpoint from a measured node-box anchor. By default, anchored paired edges use two smooth cubic halves through the edge layout point while preserving the endpoint anchor tangents. Set `route: "hobby-through"` to instead build a single Hobby spline through the source anchor, source guide, edge point, sink guide, and sink anchor, then split source/sink styling at the edge point. `route: "direct"` keeps the same anchored cubic routing but suppresses the default edge-position Hobby route. Half-edge `route-points` are followed by default on ordinary, anchored, and dangling edges using a Hobby spline. They are ordered from each node toward the edge layout point. Set `route-points: "ignore"` on both halves to suppress them. The source/sink style split remains at the edge point. `route: "straight-through"` draws the two straight force springs from source to edge position and from edge position to sink. `pattern-natural-endpoints` defaults to `false`. For built-in coil strings (`"coil"`, `"helix"`, `"spring"`) on complete continuous paths with both endpoints anchored, `true` constructs a fitted `kurvst.coil` dictionary in Typst and applies it once via ordinary `kurvst.pattern`: full-amplitude inward endpoint phases and half-integer coil periods, with exact endpoints and no taper, straight stubs, or connectors. This overrides `pattern-phase`, bypasses `pattern-fit` integer fitting, and ignores `pattern-endpoint-slope` (`endpoint-ramp: false`). Fitted dictionaries also work directly as `pattern` without this flag; see the Kurvst manual's Path Patterns section for the fitting formula and one-pass application settings. The gluon preset in `examples/map-style.typ` enables this option. Setting it to `false` restores the 75%-wavelength endpoint taper (capped at half the path length), with a squared longitudinal envelope. Without automatic fitting, `pattern-fit: true` adjusts the wavelength to the nearest whole number of periods on a complete, unbroken edge. `pattern-phase` is in radians; `calc.pi / 2` gives coils matching, gently tapered endpoint phases without straight end sections. `pattern-endpoint-slope` sets the taper's initial slope, from 0 to 3. Zero (default) keeps tangential ends; positive values allow angled ends without detaching them. The angle also depends on phase, amplitude, and wavelength. Partially anchored paths, including split-style halves and crossing-gap fragments, keep the requested wavelength, taper only anchored endpoints, and preserve phase continuity across hidden spans. Neither automatic coil construction nor integer fitting applies. A finite layer can set `shift` to move along the logical edge, with positive values moving toward its end. `label` attaches content near the layer midpoint; `label-shift` (default `0`) moves its reference point by signed arc length on that derived path, clamped to the path's endpoints. Positive values move toward its end without moving or trimming the layer. Automatic labels slide along the path and may switch sides to reduce overlaps while preserving `label-gap`; `label-shift` sets their preferred position. Set `label-slide: false` to keep that exact position and side. Explicit anchors also stay fixed. `label-path` can attach a short path (for example a momentum arrow) to a full-length invisible label carrier. Its dictionary uses ordinary path style options: offset, length, ratio, shift, stroke and mark. Arrow and label share the chosen arc displacement and side; their configured relative shift is preserved. Automatic choices keep the complete arrow inside the carrier. Disabling label sliding pins both; hiding the label leaves the arrow at its configured position. `label-side` is `auto`, `"left"`, `"right"`, or a signed number; `auto` prefers the layout's original side but can flip to avoid overlaps. Explicit side choices stay fixed while the label slides. If a label-side offset has no layout direction yet, both offset and label follow the sign of `offset` (positive means left). With `label-style.anchor` omitted or set to `auto` (also

sink-style `dictionary` or `array` or `function` or `none`

Sink half-edge style dictionary, array of layer dictionaries, or callback. A callback receives edge data. `mark-position: "center-if-dangling"` has the same dangling-edge behavior as for `source-style`, and `mark-orientation: "edge"` participates in the same orientation-aware mark placement. `sink-anchor` may be a CeTZ anchor name such as `"north"` or `"south"` to route this endpoint into a measured node-box anchor. `route: "direct"` has the same meaning as in `source-style`.

Default: `(:)`

edge-label `content` or `string` or `function` or `none`

Edge label content or callback. A callback receives edge data.

Default: `none`

edge-label-style `dictionary` or `function` or `none`

Edge-label style forwarded to `cetz.draw.content`; use `anchor` to choose which point of the label is placed at the layout label position. A callback receives edge data.

Default: `(:)`

edge-omega `float`

Hobby curl used at the endpoints of paired edge curves.

Default: `1.0`

edge-trim-accuracy `float`

Optional style key for anchored source/sink routes. Set `anchor-control-distance` in `source-style` or `sink-style` to override that endpoint's automatic guide distance in graph units. Per-edge endpoint styles control each end independently; `auto` is resolved independently. Set it in `edge-style` to supply a shared distance for both ends. -> auto | int | float Arc-length accuracy for trimming edge curves at node outsets.

Default: `0.001`

padding `none` or `int` or `float` or `array` or `dictionary`

CeTZ canvas padding.

Default: `0.4`

debug-edge-radius `int` or `float`

Radius for edge-position markers shown at `debug >= 2`.

Default: `0.08`

debug-edge-fill `any`

Fill for edge-position markers shown at `debug >= 2`.

Default: `rgb("#ff9f1c")`

debug-edge-stroke `any`

Stroke for edge-position markers shown at `debug >= 2`.

Default: `rgb("#d72638") + 0.35pt`

debug-edge-label-fill `any`

Label fill for edge-position markers shown at `debug >= 2`.

Default: `rgb("#7a1020")`

subgraph-edge-style `dictionary`

Default CeTZ style used to shade half-edges included in `subgraph`. Individual subgraph records may override this with `edge-style`.

Default: `(stroke: rgb("#ffd166") + 4.5pt)`

subgraph-edge-underlay `bool`

Draw subgraph shading below the normal half-edge style.

Default: `true`

subgraph

- label()
- bits()
- compass()
- to-label()
- hedges()
- contains()
- select()
- with-data()
- hedge-data()
- complement()
- contains-edge()
- node-depths()

label

Construct a subgraph object from a base62 label.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>, compass: "e"), graph.sink(<b>))
})
#let east = subgraph.compass(g, "e")
#let same = subgraph.label(g, subgraph.to-label(east))
#subgraph.hedges(same).len()
```

Parameters

```
label(
  graph: dictionary,
  label: string
) -> dictionary
```

graph `dictionary`

Graph object whose half-edge set the label refers to.

label `string`

Base62 subgraph label returned by `to-label` or produced by Linnest.

bits

Construct a subgraph object from a boolean hedge array.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>), graph.sink(<b>))
})
#let first = subgraph.bits(g, (true, false))
#subgraph.contains(first, 0)
```

Parameters

```
bits(
  graph: dictionary,
  bits: array
) -> dictionary
```

graph `dictionary`

Graph object whose half-edge order defines the bit array.

bits `array`

Boolean array selecting half edges by graph half-edge index.

compass

Construct a subgraph object from a DOT compass point such as `"n"` or `"s"`.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>, compass: "e"), graph.sink(<b>))
})
#subgraph.hedges(subgraph.compass(g, "e")).len()
```

Parameters

```
compass(
  graph: dictionary,
  compass: string
) -> dictionary
```

graph `dictionary`

Graph object to filter by half-edge compass statement.

compass `string`

DOT compass point such as `"n"`, `"s"`, `"e"`, or `"w"`.

to-label

Convert a subgraph object to its base62 label.

The label encodes only membership, not topology or annotations.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>), graph.sink(<b>))
})
#subgraph.to-label(subgraph.bits(g, (true, false)))
```

Parameters

```
to-label(subgraph: dictionary) -> string
```

subgraph `dictionary`

Subgraph object returned by this module or by `graph.cycles`/`graph.forests`.

hedges

Return the hedge indices included in a subgraph object.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>), graph.sink(<b>))
})
#subgraph.hedges(subgraph.bits(g, (true, false)))
```

Parameters

```
hedges(subgraph: dictionary) -> array
```

subgraph `dictionary`

Subgraph object to inspect.

contains

Test whether a subgraph object includes a hedge.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>), graph.sink(<b>))
})
#subgraph.contains(subgraph.bits(g, (true, false)), 0)
```

Parameters

```
contains(
  subgraph: dictionary,
  hedge: int
) -> bool
```

subgraph `dictionary`

Subgraph object to inspect.

hedge `int`

Half-edge index to test.

select

Select the union of node incidences, whole edges, edge halves, and exact hedges.

Node and edge references are Typst label names or numeric IDs. `source` and `sink` refer to the structural halves, independently of drawing orientation. Missing references or absent requested halves are errors. An isolated node contributes no hedges: subgraphs represent half-edge sets, not node sets.

```
#let g = graph.build({
  graph.node(<a>)
  graph.edge(<D1>, graph.source(<a>))
  graph.edge(<D6>, graph.sink(<a>))
})
#let left = subgraph.select(g, source: (<D1>,), sink: (<D6>,))
#subgraph.hedges(left)
```

Parameters

```
select(
  g: dictionary,
  nodes: array,
  edges: array,
  source: array,
  sink: array,
  hedges: array
) -> dictionary
```

g `dictionary`

Graph whose records define names and IDs.

nodes `array`

Select all half edges incident to these nodes.

Default: `()`

edges `array`

Select both available halves of these edges.

Default: `()`

source `array`

Select only the source half of these edges.

Default: `()`

sink `array`

Select only the sink half of these edges.

Default: `()`

hedges `array`

Select these exact numeric half-edge IDs.

Default: `()`

with-data

Attach arbitrary Typst data without serializing it through Wasm.

`data: auto` preserves subgraph-wide data; `hedge: none` preserves annotations. A hedge callback receives the original endpoint record plus numeric `edge`, label-valued `edge-name` (or `none`), and `flow` (`"source"` or `"sink"`). Its `data` is the existing subgraph annotation; `graph-data` holds the original half-edge data. The return value replaces the annotation, including `none`. Other non-function hedge values replace every selected hedge's annotation. Only selected hedges retain annotations; callback values are never evaluated unless supplied as the `hedge` updater itself.

Parameters

```
with-data(
  g: dictionary,
  s: dictionary,
  data: any,
  hedge: any
) -> dictionary
```

g `dictionary`

Compatible graph supplying endpoint records and graph data.

s `dictionary`

Subgraph to annotate.

data `any`

Replacement subgraph-wide data, or `auto` to preserve it.

Default: `auto`

hedge `any`

Half-edge updater, constant annotation, or `none` to preserve annotations.

Default: `none`

hedge-data

Return a selected hedge's annotation, or `none` when it has no annotation. Unselected and out-of-range half-edge IDs are errors.

Parameters

```
hedge-data(
  s: dictionary,
  h: int
) -> any
```

s `dictionary`

Subgraph whose annotation is read.

h `int`

Selected numeric half-edge ID.

complement

Return the complement of a subgraph's selected half edges.

Subgraph-wide data is preserved; newly selected hedges have no annotations.

Parameters

```
complement(
  graph: dictionary,
  selected: dictionary
) -> dictionary
```

graph `dictionary`

Graph object whose half-edge order defines the result.

selected `dictionary`

Subgraph object to invert.

contains-edge

Test whether either half-edge of an edge record is selected.

Parameters

```
contains-edge(
  subgraph: dictionary,
  edge: dictionary
) -> bool
```

subgraph `dictionary`

Subgraph object to inspect.

edge `dictionary`

Edge record from `graph.edges(...)` or draw callback data.

node-depths

Compute breadth-first node depths inside a selected edge set.

Parameters

```
node-depths(
  graph: dictionary,
  selected: dictionary,
  root: int
) -> dictionary
```

graph `dictionary`

Graph object to inspect.

selected `dictionary`

Subgraph whose selected half edges define the traversal edges.

root `int`

Root node index.

Default: `0`